

AUTOR(A) DA DISSERTAÇÃO

Título da Dissertação



UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE MATEMÁTICA

2019

AUTOR(A) DA DISSERTAÇÃO

Título da Dissertação

Dissertação apresentada ao Programa de Pós-
Graduação...

UBERLÂNDIA - MG
2010

Dedicatória.

Agradecimientos

Agradecimientos.

Resumo

Resumo em português.

foi quando a vida me deu um doce

Lista de Figuras

1.1	Organização da Seção 1.1.	14
1.2	Estrutura de um neurônio típico.	15
1.3	Modelo de uma unidade de McCulloch e Pitts.	16
1.4	Gráficos das Funções de Ativação.	18
1.5	Representação da taxa de aprendizagem.	20
1.6	Organização da Seção 1.2.	21
1.7	Estrutura visual básica de uma unidade e uma rede neural com duas camadas ocultas.	22
1.8	Arquiteturas de Redes Neurais.	24
1.9	Ilustração do processo de <i>backpropagation</i> em uma rede neural.	25
1.10	Comparação entre o método de descida do gradiente tradicional e o Gradiente Descendente Estocástico.	27
1.11	Ilustração do <i>Overfitting</i>	28
1.12	Ilustração do <i>Dropout</i> removendo 60% das unidades aleatoriamente durante uma época de treinamento.	30
1.13	Ilustração do <i>Early Stopping</i>	30
1.14	Gráfico da derivada da função sigmoid logística.	32
1.15	Exemplos de dígitos manuscritos do conjunto de dados MNIST.	33
1.16	Arquitetura da rede neural para classificação de dígitos manuscritos. Inicialmente, é enviado o número de pixels da imagem (784) para a camada de entrada (unidade $x_{784} = 0,4$ intensidade do pixel), que é processada por uma camada oculta com 30 unidades, e a camada de saída tem 9 unidades, correspondendo às 10 classes de dígitos (0 a 9).	33
1.17	Fluxo de dados durante o processo de treinamento e avaliação da rede neural.	34
1.18	Organização da Seção 1.3.	35
1.19	Exemplo de uma imagem/matriz 10×10 em escala de cinza, em que cada pixel tem um valor de intensidade entre 0 (preto) e 255 (branco).	36
1.20	Ilustração da operação de convolução discreta. O kernel w é deslizado sobre o vetor x , e em cada posição, a soma ponderada dos valores de x cobertos pelo kernel é calculada para gerar o valor correspondente na saída.	40

1.21	Exemplo de convolução bidimensional. A matriz de entrada X (matriz 7×7) é convoluída com o <i>kernel</i> W (matriz 2×2) para produzir a saída S (matriz 6×6). O <i>kernel</i> é “deslizado” sobre a matriz de entrada, formando sub-matrizes, e em cada posição, a soma ponderada dos valores dos pixels cobertos pelo <i>kernel</i> é calculada e o resultado é aplicado em uma função de ativação (σ) para gerar o valor correspondente ($\hat{y}_{11}$) na matriz de saída. b sendo o viés adicionado à soma ponderada.	42
1.22	Aplicações de diferentes <i>kernels</i> em uma imagem. A imagem original é representada em (a), juntamente com o <i>kernel</i> identidade, que não faz modificações. Em (b) a imagem passou por um <i>kernel</i> que aumenta a nitidez reforçando o pixel central e aumentando a diferença dele em relação à sua vizinhança. Em (c) foi usado um <i>kernel</i> de desfoque, que substitui um pixel pela média dele com seus vizinhos. A imagem em (d) é o resultado de aplicar na imagem em escala de cinza um <i>kernel</i> que reforça suas linhas verticais, enquanto (e) reforça suas linhas horizontais. O <i>kernel</i> em (f) reforça os contornos (bordas) da imagem. . .	42
1.23	Aplicação de <i>padding</i> em uma imagem. Em (a) a imagem original passou por zero <i>padding</i> . Em (b) a imagem original passou por <i>reflection padding</i> . A imagem em (c) passou por um <i>padding</i> constante, semelhante em (a), mas com valor diferente de zero.	44
1.24	Exemplo de Max Pooling com tamanho de 2×2 . A imagem original é reduzida para a imagem menor após a aplicação do Max Pooling. Cada bloco de 2×2 pixels na imagem original é substituído pelo valor máximo dentro desse bloco na imagem reduzida.	45
1.25	Arquitetura típica de uma Rede Neural Convolucional com uma camada de convolução e uma camada de <i>pooling</i> , em que é enviado para rede a imagem de um cachorro e é feita a classificação do objeto na imagem.	47
1.26	Exemplo da operação de convolução transposta.	48
1.27	Imagem de entrada X e resposta desejada Y	49
1.28	Aplicação de um <i>kernel</i> de convolução na imagem de entrada X . O resultado da convolução é um mapa de características que destaca as regiões de interesse na imagem.	50
1.29	Aplicação de padding na imagem de entrada X para preservar suas dimensões durante a operação de convolução.	50
1.30	Aplicação da função de ativação ReLU no mapa de características resultante da convolução.	51
1.31	Aplicação de uma camada de <i>Max pooling</i> no mapa de características resultante.	51
1.32	Aplicação da operação de convolução transposta no mapa de pooling para expandir suas dimensões espaciais e voltar às dimensões originais da imagem de entrada.	52

1.33	Estrutura em forma de U da arquitetura U-Net. O encoder é responsável por extrair características da imagem de entrada, enquanto o decoder reconstrói a imagem, utilizando as características extraídas pelo encoder.	52
1.34	Exemplo de conexões de atalho (<i>skip connections</i>) na arquitetura U-Net.	53
1.35	Exemplo de conexões de atalho (<i>skip connections</i>) na arquitetura U-Net. As conexões de atalho permitem que as informações de baixo nível, como bordas e texturas, sejam diretamente transferidas do encoder para o decoder, facilitando a reconstrução precisa da imagem segmentada.	54
1.36	Aplicação de uma última camada de convolução para gerar a imagem de saída Y . O <i>kernel</i> utilizado tem dimensões $(1 \times 1 \times 2)$, um canal para cada imagem resultante da concatenação.	54
1.37	Aplicação de um limiar (<i>threshold</i>) para gerar a imagem binária final. A imagem segmentada resultante da aplicação da função de ativação sigmoid tem valores entre 0 e 1, representando a probabilidade de cada pixel pertencer à classe segmentada. O limiar é aplicado para converter esses valores em uma imagem binária, onde pixels com valores acima do limiar são classificados como pertencentes à classe segmentada (1), enquanto pixels com valores abaixo do limiar são classificados como pertencentes ao plano de fundo (0).	55
1.38	Arquitetura completa da U-Net.	56
2.1	Operações entre conjuntos fuzzy.	60
2.2	Arquitetura de um Sistema Baseado em Regras Fuzzy (SBRF).	64
2.3	Exemplo de inferência fuzzy utilizando o método de Mamdani.	65

Lista de Tabelas

1.1	Banco de dados com as avaliações dos colegas e a decisão final de compra.	18
-----	---	----

Sumário

Agradecimentos	4
Resumo	5
Lista de Figuras	9
Lista de Tabelas	10
Introdução	13
1 Fundamentação Teórica	14
1.1 Neurônio Biológico e Artificial	14
1.1.1 Neurônio Biológico	15
1.1.2 Unidade	16
1.1.3 Tipos de Função de Ativação	16
1.1.4 Perceptron e Algoritmo de Aprendizagem	18
1.2 Redes Neurais	21
1.2.1 Arquiteturas de Redes Neurais	23
1.2.2 <i>Backpropagation</i>	24
1.2.3 Gradiente Descendente Estocástico	26
1.2.4 <i>Overfitting</i>	27
1.2.5 Hiperparâmetros	28
1.2.6 Regularização	29
1.2.7 <i>Momentum</i>	30
1.2.8 <i>Vanishing Gradient</i>	31
1.2.9 Dígitos Manuscritos	32
1.3 Rede Neural para Análise de Imagens	35
1.3.1 Córtex Visual Humano e Visão Computacional	35
1.3.2 Redes Neurais Convolucionais	37
1.3.3 Convolução	37
1.3.4 Correlação Cruzada	43
1.3.5 <i>Kernel</i> Multicamadas	43
1.3.6 <i>Padding</i>	44

1.3.7	<i>Stride</i>	44
1.3.8	<i>Pooling</i>	45
1.3.9	Convolução Transposta	46
1.4	U-net: Rede Convolutacional para Segmentação de Imagens	49
1.4.1	U-Net	55
1.4.2	DICE	57
2	Teoria dos Conjuntos Fuzzy	58
2.1	Teoria dos Conjuntos Fuzzy	58
2.1.1	Conjuntos Fuzzy	58
2.1.2	Operações entre Conjuntos Fuzzy	59
2.1.3	Normas Triangulares	59
2.1.4	Níveis de um Conjunto Fuzzy	61
2.1.5	Números Fuzzy	62
2.2	Sistema Baseado em Regras Fuzzy	62
2.2.1	Regras e Inferência Fuzzy	63
2.2.2	Sistema Baseado em Regras Fuzzy	63

Introdução

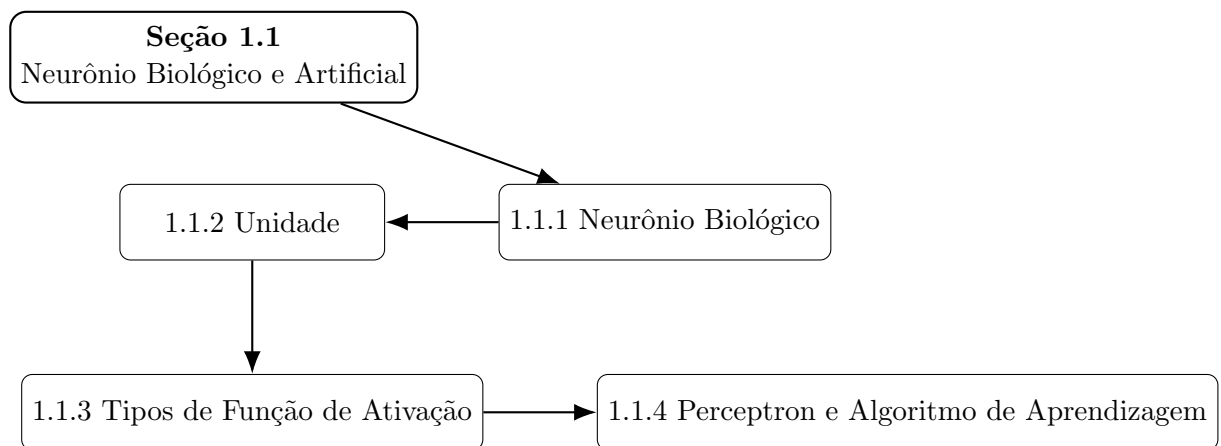
Helloo baby

Capítulo 1

Fundamentação Teórica

1.1 Neurônio Biológico e Artificial

Figura 1.1: Organização da Seção 1.1.



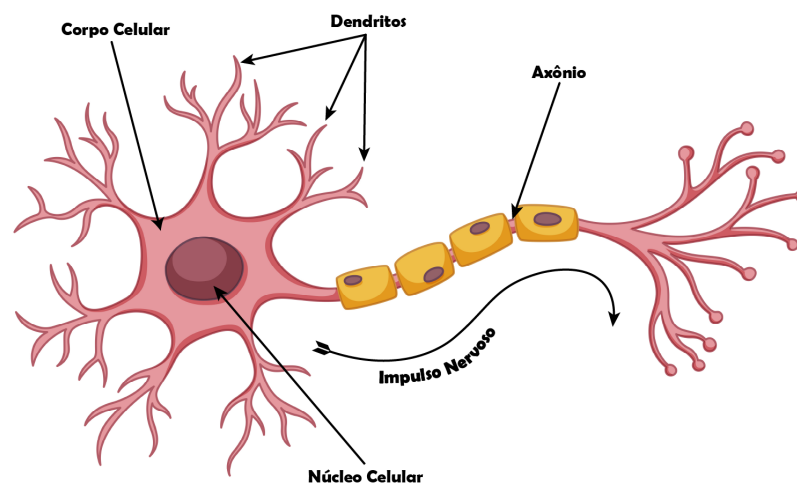
Fonte: Elaboração do Autor.

Na área de Inteligência Artificial, comumente utiliza-se o termo *neurônio artificial* para se referir a uma unidade de processamento em um modelo de rede neural. O neurônio artificial é inspirado no funcionamento do neurônio biológico, que é a unidade básica do sistema nervoso. Assim como os neurônios biológicos, os neurônios artificiais recebem entradas, processam essas entradas e produzem uma saída. A principal função de um neurônio artificial é realizar uma combinação linear das entradas, seguida por uma função de ativação não linear, que determina a saída do neurônio. A semelhança entre esses dois objetos (Neurônio artificial/Neurônio Biológico) só está na inspiração, sendo bem diferentes em seu funcionamento, com isso, neste trabalho utiliza-se o termo *unidade* (O'SHEA; NASH, 2015) para se referir ao neurônio artificial. Os conceitos apresentados a seguir são fundamentais para o entendimento do funcionamento das redes neurais, que serão abordados na seção 1.2.

1.1.1 Neurônio Biológico

Para entender o funcionamento da unidade matemática é necessário entender o funcionamento do neurônio biológico. Neurônios são “células excitáveis” que se comunicam ao transmitir impulsos elétricos. Células *excitáveis* são definidas como células capazes de produzir sinais elétricos amplos e rápidos denominados *potências de ação* (veja (STANFIELD, 2014)). A maioria dos neurônios contém três componentes principais: um corpo celular, o(s) dendrito(s) e um axônio. O corpo celular, chamado de *soma* contém o núcleo celular, local onde parte da informação recebida de outras partes do corpo é manipulada. Os *dendritos* se ramificam a partir do corpo celular, são responsáveis por receber e integrar uma imensa quantidade de sinais provenientes de outros neurônios. Outra ramificação advinda do corpo celular é o *axônio*. Diferente dos dendritos, cuja função é *receber* informação, um axônio *envia* informações. Grande parte dos neurônios tem apenas um axônio, mas os axônios conseguem se ramificar, enviando, assim, sinais a mais de um destino. Esses três componentes, de forma resumida, formam o neurônio biológico, como mostrado na Figura 1.2.

Figura 1.2: Estrutura de um neurônio típico.



Fonte: Adaptado de (STANFIELD, 2014).

Outro fator importante em um neurônio biológico são os potenciais de ação, gerados nos dendritos, são avaliados e ponderados por meio da *força sináptica* (BEAR; CONNORS; PARADISO, 2017), que nos diz o quão importante a informação. Após esse processo, são enviados para o núcleo celular. Entre o corpo celular e o axônio será decidido se o potencial é excitatório ou inibitório. A decisão é feita caso o potencial de ação supere ou não um *limiar*. Caso seja excitatório, a informação é passada a diante, caso seja inibitório, a informação é suprimida.

1.1.2 Unidade

A partir das informações obtidas com o estudo do neurônio biológico, pesquisadores tentaram simular seu funcionamento em computador. A implementação mais bem aceita de simulação foi proposta por dois pesquisadores, o neurobiologista Warren McCulloch e o matemático Walter Pitts em 1943 em (MCCULLOCH; PITTS, 1943) que utilizaram-se da lógica proposicional para descrever o funcionamento do sistema nervoso central, Figura 1.3. McCulloch acreditava que o neurônio biológico tinha uma natureza binária, ou seja, um neurônio influencia outro ou não, entretanto, posteriormente essa crença mostrou-se errônea com os avanços de estudos na área.

As variáveis $x_1, x_2, x_3, \dots, x_n$ são denominadas *dados de entrada*, em analogia às sinapses de um neurônio biológico. Os valores $w_1, w_2, w_3, \dots, w_n$ são os *pesos* associados a cada dado de entrada, os pesos nos dizem o quão importante é a entrada para a *soma*, parte central em laranja na Figura 1.3. O viés tem a mesma funcionalidade do limiar no neurônio biológico, determinando se o neurônio será ativado ou não. Para resumir a notação, defini-se o viés de b (do inglês, *bias*). Na soma, a variável z é obtida fazendo a combinação linear das variáveis:

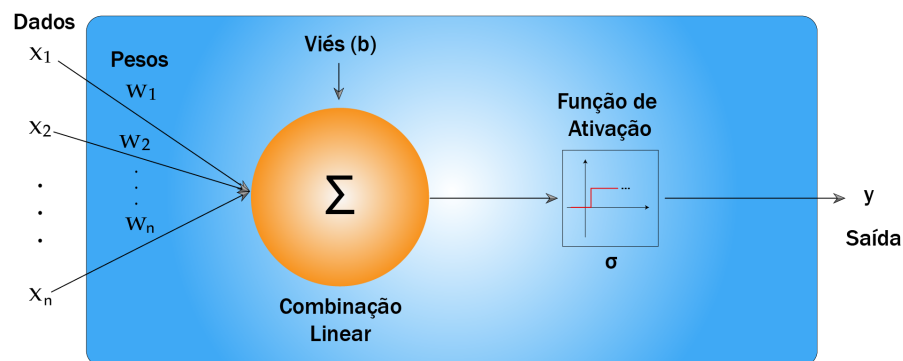
$$x_1 \times w_1 + x_2 \times w_2 + x_3 \times w_3 + \dots + x_n \times w_n + b = z = \sum_{i=1}^n x_i \times w_i + b.$$

Em seguida, o valor z é aplicado a uma *função de ativação* $\sigma : \mathbb{R} \rightarrow \mathbb{R}$, resultando na saída $\hat{y}$, ou seja:

$$\hat{y} = \sigma \left(\sum_{i=1}^n w_i x_i + b \right). \quad (1.1)$$

mudar imagem

Figura 1.3: Modelo de uma unidade de McCulloch e Pitts.



Fonte: Elaboração do Autor.

1.1.3 Tipos de Função de Ativação

A função de ativação, representada por σ , define a saída de uma unidade, pode-se definir diferentes tipos de funções de ativação, como a função de limiar, função linear por partes e a

função sigmoidal.

- Função de Limiar: Para este tipo de função de ativação, descrito na equação (1.2), a unidade é ativada (ou seja, produz uma saída de 1) se a soma ponderada das entradas mais o viés for maior ou igual a zero. Caso contrário, a saída é 0.

$$\sigma(z) = \begin{cases} 1, & \text{se } z \geq 0 \\ 0, & \text{se } z < 0 \end{cases}. \quad (1.2)$$

- Função Sigmoidal: A função sigmoidal, cujo o gráfico tem a forma de um “S”, é de longe a forma mais comum de função de ativação utilizada na construção de unidades. Esta é definida como uma função estritamente crescente que exhibe um balanceamento adequado entre comportamento linear e não linear. Um exemplo de função sigmoidal é a *sigmoid logística*, definida por:

$$\sigma(z) = \frac{1}{1 + \exp(-\alpha z)}, \quad (1.3)$$

em que α é um parâmetro que controla a inclinação da curva. A função sigmoid logística tem um conjunto de propriedades que se mostrarão muito úteis nos cálculos relacionados à aprendizagem dos pesos:

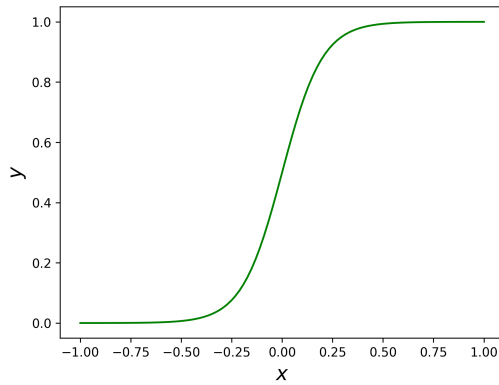
1. Não linear;
 2. Contínua e diferenciável em todo o domínio de $\mathbb{R}$;
 3. A derivada tem forma simples e é expressa pela própria função: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$;
 4. É estritamente monótona, ou seja, $z_1 \leq z_2 \iff \sigma(z_1) \leq \sigma(z_2)$.
- Função *Rectified Linear Unit* (ReLU): A função ReLU (do inglês, Unidade Linear Retificada) é definida como $\sigma(z) = \max(0, z)$. Esta é amplamente utilizada em redes neurais profundas (estudada na Seção 1.2) devido à sua simplicidade computacional. A função ReLU ativa a unidade apenas quando a soma ponderada das entradas mais o viés é positiva, caso contrário, a saída é zero, ou seja:

$$\sigma(z) = \begin{cases} z, & \text{se } z > 0 \\ 0, & \text{se } z \leq 0 \end{cases}. \quad (1.4)$$

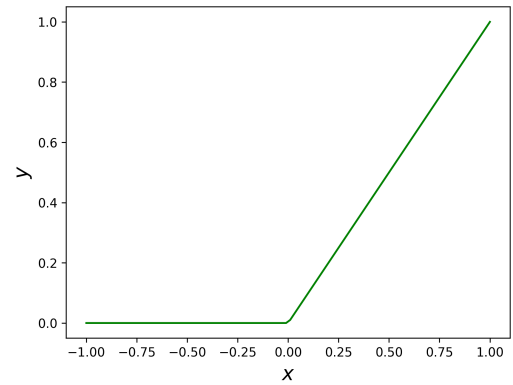
Em geral, o objetivo da função de ativação é limitar a saída de uma unidade para um intervalo de valores, na Figura 1.4a é mostrado o gráfico da função sigmoid logística, enquanto na Figura 1.4b é mostrado o gráfico da função *Rectified Linear Unit*.

A principal desvantagem na unidade de McCulloch e Pitts é a ausência de um algoritmo de aprendizagem, ou seja, uma maneira de ajustar os pesos e o viés da unidade para que este seja capaz de resolver um problema específico. Essa limitação foi superada em 1958 por Frank

Figura 1.4: Gráficos das Funções de Ativação.



(a) Função Sigmoid.



(b) Função ReLU.

Fonte: Elaboração do Autor.

Rosenblatt, que propôs o *Perceptron* (ROSENBLATT, 1958), um modelo de unidade com um algoritmo de aprendizagem capaz de ajustar os pesos e o viés da unidade.

1.1.4 Perceptron e Algoritmo de Aprendizagem

O Perceptron é um modelo de unidade que utiliza um algoritmo de aprendizagem supervisionada para ajustar seus pesos e viés com base em um conjunto de dados “rotulados”¹. O objetivo do Perceptron é encontrar uma função que mapeie corretamente as entradas para as saídas desejadas, minimizando o erro entre as previsões do modelo e os valores reais.

Para esclarecer o funcionamento do Perceptron e do algoritmo de aprendizagem, é proposto o Exemplo 1.1.

Exemplo 1.1. *Um investidor busca decidir qual ação adquirir com base nas variáveis: o valor é caro?, é confiável? e é volátil?. Para isso, dispõe das avaliações de quatro colegas que também analisam essas mesmas características antes de realizar seus investimentos.*

Tabela 1.1: Banco de dados com as avaliações dos colegas e a decisão final de compra.

	valor	confiança	volatilidade	y
x_1	0	1	0	1
x_2	1	0	1	0
x_3	1	1	1	0
x_4	0	0	0	1

Na Tabela 1.1 são apresentadas as avaliações dos colegas e a decisão final de compra em que 0 representa “não”, 1 representa “sim”, $x_1, \dots, x_4$ representam as respostas de cada colegas e y a decisão final de cada um. Com isso, o Perceptron terá três variáveis de entrada e como saída, a decisão sobre a compra da ação. Os pesos iniciais são definidos aleatoriamente como:

¹Dados rotulados são conjuntos de dados de entrada associados a saídas desejadas, ou seja, para cada conjunto de entradas, há uma saída correta conhecida.

$w_1 = w_2 = w_3 = 0$ e o viés como $b = 0$. A função de ativação utilizada é a função ReLU (1.4). Testando o primeiro dado de entrada $x_1 = (0, 1, 0)$ e $y = 1$, obtém-se:

$$z = x_1 \cdot w_1 + b = 0 \Rightarrow \hat{y} = \sigma(0) = 0,$$

o símbolo $\cdot$ representa o produto escalar entre os vetores. Como $y = 1 \neq \hat{y} = 0$, em que $\hat{y}$ é a resposta da rede definido como na equação (1.1). O Perceptron comete um erro e é necessário aplicar o algoritmo de aprendizagem (a unidade de McCulloch e Pitts pararia aqui). Para isso, define-se uma função de custo, responsável por quantificar o erro cometido durante o treinamento. Neste exemplo, utiliza-se o Erro Quadrático Médio, dado por:

$$C(w_i, b_i) = \frac{\sum_{i=1}^k (y_i - \hat{y}_i)^2}{n}, \quad (1.5)$$

em que y_i é a resposta do dado de entrada x_i e n representa a quantidade de dados presentes no conjunto de treinamento. $C(w_i, b_i)$ não depende de w_i e b_i explicitamente, mas sim por meio de $\hat{y}_i$, ou seja, $C(w_i, b_i) = C(\hat{y}_i(w_i, b_i))$, já que $\hat{y}_i = \sigma(z_i)$ e $z_i = x_i \cdot w_i + b_i$. No caso do Exemplo 1.1, x_i e w_i são vetores.

O processo de treinamento ajusta iterativamente os pesos e o viés por meio do método da Descida do Gradiente (SG - *Gradient Descent*), cujo objetivo é minimizar a função de custo. Esse método utiliza as derivadas parciais da função de custo em relação aos pesos e ao viés, o que indica a direção de maior crescimento da função. Para reduzir o erro, atualizam-se os parâmetros na direção oposta ao gradiente. As derivadas são:

$$\frac{\partial C(w_i, b_i)}{\partial w_i} = -2 \cdot \frac{\sum_{i=1}^k (y_i - \hat{y}_i) \cdot \sigma'(z_i) \cdot x_i}{n}; \quad (1.6)$$

$$; \frac{\partial C(w_i, b_i)}{\partial b_i} = -2 \cdot \frac{\sum_{i=1}^k (y_i - \hat{y}_i) \cdot \sigma'(z_i)}{n}. \quad (1.7)$$

No o Exemplo 1.1 assume-se $\sigma'(z) = 1$ para simplificação. Como a derivada aponta para a região de maior valor da função, multiplica-se por -1 para obter a direção da região de minimização. Com isso, é obtido o Algoritmo 1.

Entrada: Taxa de aprendizado η , função de custo $C(w, b)$, pesos iniciais

$w_1, w_2, \dots, w_n$ e viés b

Saída: Pesos e viés ajustados

Inicializar os pesos w_i e o viés b com valores pequenos aleatórios;

Repita

 Calcular a saída do modelo;

$\hat{y}_i \leftarrow \sigma(\sum_{i=1}^n w_i \cdot x_i + b)$;

 Calcular o erro;

$\Delta_w \leftarrow \frac{\sum_{i=1}^n (y_i - \hat{y}_i) \cdot \sigma'(z_i) \cdot x_i}{n}$;

$\Delta_b \leftarrow \frac{\sum_{i=1}^n (y_i - \hat{y}_i) \cdot \sigma'(z_i)}{n}$;

 Atualizar os pesos e o viés;

$w_i \leftarrow w_i + \eta \Delta_w$, para todo $i = 1, 2, \dots, n$;

$b \leftarrow b + \eta \Delta_b$;

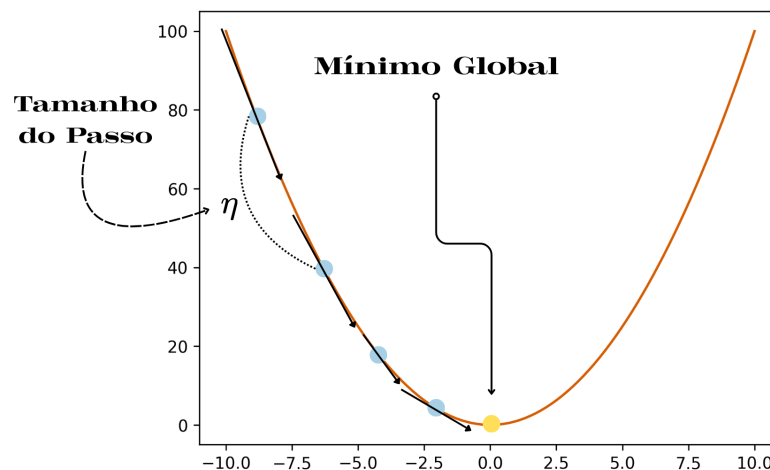
até convergência ou número máximo de iterações;

Retorne w_i, b ;

Algoritmo 1: Método da Descida do Gradiente.

A taxa de aprendizagem (η) presente no Algoritmo 1 pode ser interpretada como o tamanho do passo dado em direção ao mínimo da função de custo, como mostra a Figura 1.5. Quando o valor dessa taxa é muito pequeno, o processo de convergência torna-se mais lento, exigindo um número maior de iterações para atingir o mínimo. Por outro lado, se o passo for muito grande, o algoritmo pode ultrapassar o ponto de mínimo, dificultando ou até impedindo a convergência adequada, para o Exemplo 1.1 é utilizada uma taxa de aprendizado de 0,1.

Figura 1.5: Representação da taxa de aprendizagem.



Fonte: Elaboração do Autor.

Aplicando o Algoritmo 1 no Exemplo 1.1, os seguintes ajustes são feitos nos pesos e no viés após uma passada pelos 4 dados presentes no banco de dados:

$$w = (-0,1, 0, 1, -0,1), \quad b = 0,1.$$

Verificando as previsões da rede para os parâmetros atualizados, é obtido:

$$x_1: (0,1,0) \Rightarrow z = 0 \cdot (-0,1) + 1 \cdot 0,1 + 0 \cdot (-0,1) + 0,1 = 0,2 \Rightarrow \hat{y} = 1;$$

$$x_2: (1,0,1) \Rightarrow z = -0,1 + 0 + -0,1 + 0,1 = -0,1 \Rightarrow \hat{y} = 0;$$

$$x_3: (1,1,1) \Rightarrow z = -0,1 + 0,1 - 0,1 + 0,1 = 0 \Rightarrow \hat{y} = 0;$$

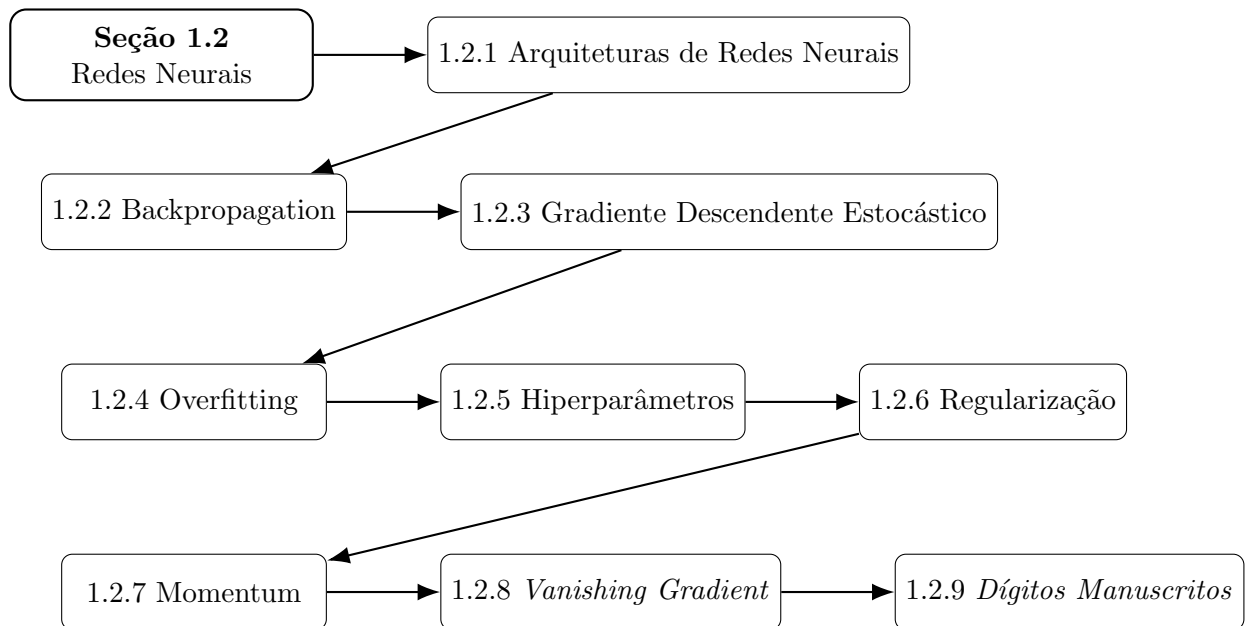
$$x_4: (0,0,0) \Rightarrow z = 0 + 0 + 0 + 0,1 = 0,1 \Rightarrow \hat{y} = 1.$$

Basta observar que $\hat{y} = \sigma(z)$ e $\sigma(z)$ é a função de ativação utilizada. Assim, após uma única época com $\eta = 0,1$ os pesos e o viés encontrados classificam corretamente todos os exemplos do conjunto de treinamento.

Comumente o banco de dados é dividido em dois conjuntos: o conjunto de treinamento e o conjunto de teste. O conjunto de treinamento é utilizado para ajustar os pesos e o viés do Perceptron, enquanto o conjunto de teste é utilizado para avaliar o desempenho do modelo em dados não vistos durante o treinamento. Essa divisão é essencial para garantir que o modelo aprenda a generalizar bem para novos dados, evitando o problema do *overfitting* (na Seção 1.2.4), que ocorre quando o modelo se ajusta excessivamente aos dados de treinamento, perdendo a capacidade de generalização.

1.2 Redes Neurais

Figura 1.6: Organização da Seção 1.2.



Fonte: Elaboração do Autor.

Nesta seção serão apresentados os conceitos relacionados às redes neurais, sendo muito importantes para o andamento do trabalho, todos os conceitos apresentados nesta seção serão utilizados na construção da rede neural proposta na Seção 1.4.

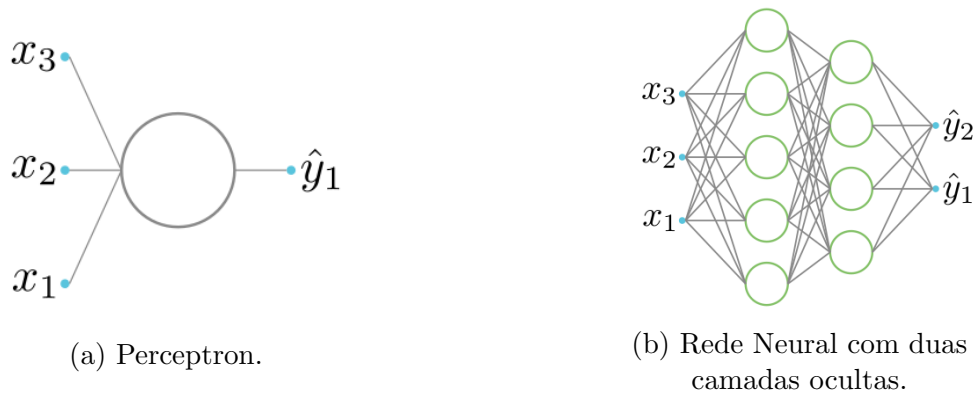
As redes neurais são um “encadeamento” de múltiplas unidades, onde a saída de uma unidade é a entrada de outra. Redes neurais podem ser organizadas em camadas: uma *camada de entrada*, uma ou mais *camadas ocultas* e uma *camada de saída*. Cada camada é composta por várias unidades e cada unidade em uma camada está conectada a todas as unidades na camada seguinte. A Figura 1.7a ilustra a estrutura básica de uma unidade vista na Seção 1.1.2. Já a Figura 1.7b mostra uma rede neural com duas camadas ocultas, em que a camada de entrada tem três unidades, a primeira camada oculta tem cinco unidades, a segunda camada oculta tem quatro unidades e a camada de saída tem duas unidades. Matematicamente, é possível representar uma rede neural com duas camadas ocultas da seguinte maneira:

$$\hat{y} = \sigma^s \left[\sum_{i=1}^4 \sigma_i^2 \left(\sum_{j=1}^5 \sigma_j^1 \left(\sum_{k=1}^3 w_{jk}^1 x_k + b_j^1 \right) w_{ij}^2 + b_i^2 \right) w_i^s + b^s \right], \quad (1.8)$$

em que σ_j^1 é a função de ativação da j -ésima unidade da primeira camada oculta, σ_i^2 é a função de ativação da i -ésima unidade da segunda camada oculta e σ^s é a função de ativação da s -ésima unidade da camada de saída. Os pesos e os vieses são representados por w_{jk}^1 , b_j^1 , w_{ij}^2 , b_i^2 , w_i^s e b^s . Generalizando a notação, para uma rede neural com L camadas ocultas, a saída da rede pode ser expressa como:

$$\hat{y} = \sigma^s \left[\sum_{i_L=1}^{n_L} \sigma_{i_L}^L \left(\sum_{i_{L-1}=1}^{n_{L-1}} \sigma_{i_{L-1}}^{L-1} \left(\cdots \sum_{k=1}^{n_1} \sigma_k^1 \left(\sum_{m=1}^{n_0} w_{km}^1 x_m + b_k^1 \right) w_{jk}^2 + b_j^2 \right) w_{ij}^L + b_i^L \right) w_i^s + b^s \right]. \quad (1.9)$$

Figura 1.7: Estrutura visual básica de uma unidade e uma rede neural com duas camadas ocultas.



Fonte: Elaboração do Autor.

Outra forma de representar uma rede neural é utilizando a notação matricial, em que as entradas, os pesos e os vieses são representados por matrizes e vetores. Para uma rede neural com n entradas, uma camada oculta com m neurônios e uma saída, a saída da rede pode ser

expressa como:

$$\underbrace{\begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix}}_{m \times n} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}}_{n \times 1} + \underbrace{\begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}}_{m \times 1}$$

Essa forma representa a saída de uma camada caso a rede tenha apenas uma camada oculta, para redes neurais com mais camadas ocultas, a notação matricial torna-se mais complexa, mas o princípio de multiplicação de matrizes e adição de vetores permanece o mesmo.

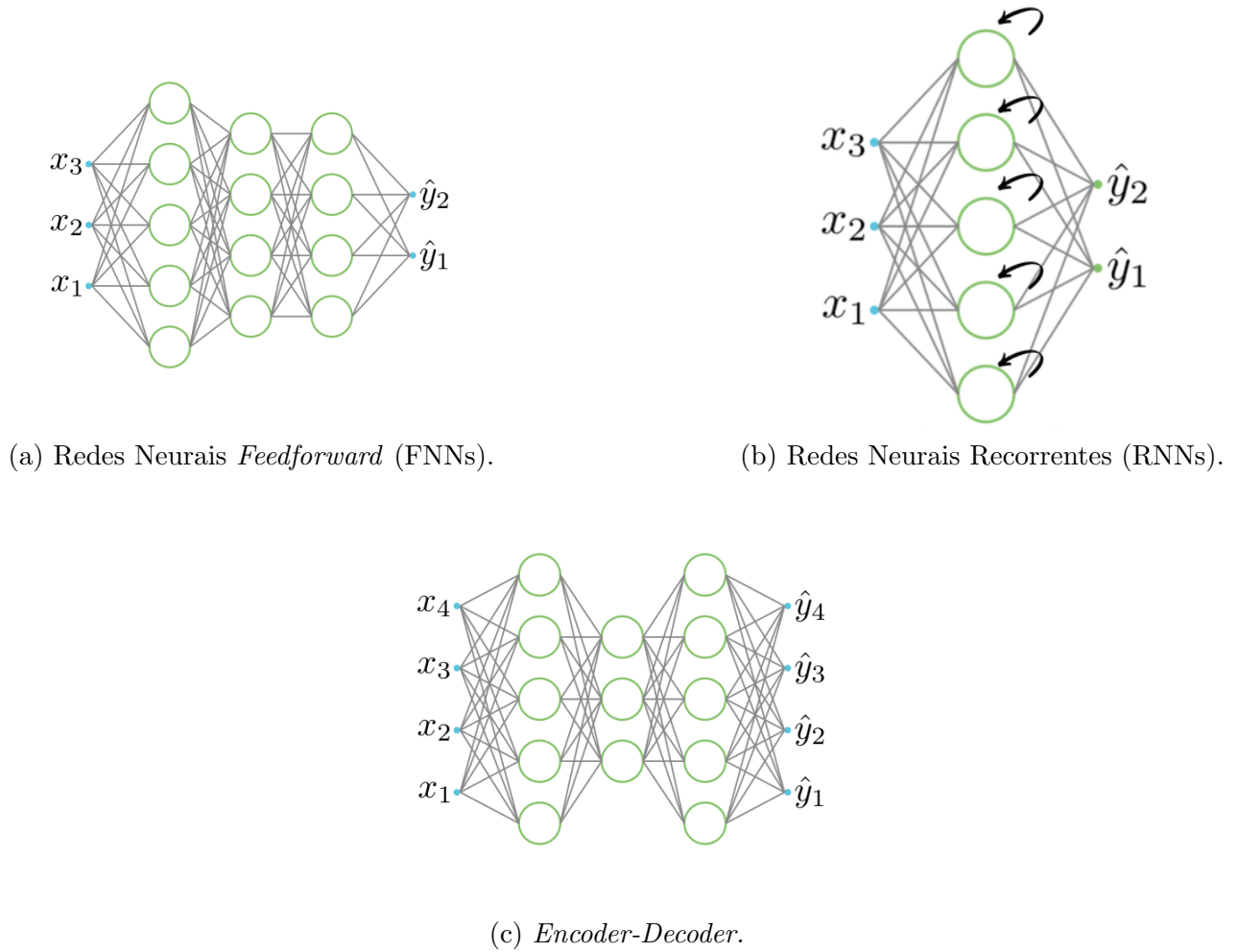
1.2.1 Arquiteturas de Redes Neurais

Existem diversas arquiteturas de redes neurais, cada uma projetada para lidar com diferentes tipos de problemas e dados. Algumas das arquiteturas mais comuns incluem ((BOOK, 2025a)):

- **Redes Neurais Feedforward (FNNs):** São as redes neurais mais simples, onde a informação flui em uma única direção, da camada de entrada para a camada de saída, passando pelas camadas ocultas. FNNs são amplamente utilizadas para tarefas de classificação e regressão.
- **Redes Neurais Convolucionais (RNCs):** São projetadas para processar dados com uma estrutura de grade, como imagens. Estas utilizam operações de convolução para extrair características espaciais dos dados de entrada. RNCs são amplamente utilizadas em tarefas de visão computacional, como reconhecimento de objetos e detecção de faces.
- **Redes Neurais Recorrentes (RNNs):** São projetadas para lidar com dados sequenciais, onde a saída em um determinado momento depende das entradas anteriores. RNNs possuem conexões recorrentes que permitem que informações sejam mantidas ao longo do tempo. Estas são amplamente utilizadas em tarefas como processamento de linguagem natural e reconhecimento de fala.
- **Redes Neurais Generativas Adversariais (GANs):** Consistem em duas redes neurais que competem entre si: um gerador e um discriminador. O gerador cria dados falsos, enquanto o discriminador tenta distinguir entre dados reais e falsos. GANs são amplamente utilizadas para geração de imagens realistas e outras tarefas criativas.
- **Encoder-Decoder:** São arquiteturas que consistem em duas partes principais: o *encoder*, que comprime os dados de entrada em uma representação latente, e o *decoder*, que reconstrói os dados a partir dessa representação. Essas redes são amplamente utilizadas em tarefas como tradução automática e geração de texto.

É possível utilizar duas arquiteturas em conjunto, como abordado na Seção 1.4.

Figura 1.8: Arquiteturas de Redes Neurais.



Fonte: Elaboração do Autor.

As Figuras 1.8a e 1.8c ilustram a arquitetura de uma Rede Neural *Feedforward*, com o funcionamento semelhante ao desenvolvido na Equação (1.9). A Figura 1.8b mostra a arquitetura de uma Rede Neural Recorrente, estas redes possuem conexões recorrentes, ou seja, a saída de uma unidade é realimentada como entrada para a mesma unidade ou para unidades anteriores, permitindo que informações sejam mantidas ao longo do tempo, por não estar no escopo deste trabalho, não serão abordados os detalhes matemáticos de funcionamento das RNNs.

1.2.2 *Backpropagation*

Assim como no perceptron, as redes neurais utilizam algoritmos de aprendizagem para ajustar seus pesos e vieses com o objetivo de minimizar uma função de custo. O algoritmo mais comum utilizado para esse propósito é o *backpropagation* (retropropagação) (HECHT-NIELSEN, 1992) e (WERBOS, 1990), que é uma extensão do método da descida do gradiente. O *backpropagation* é utilizado para calcular os gradientes da função de custo em relação aos pesos e vieses de todas as camadas da rede neural, permitindo a atualização eficiente desses parâmetros durante o treinamento. O algoritmo funciona propagando o erro da saída da rede de volta para as

camadas anteriores, ajustando os pesos com base na contribuição de cada unidade para o erro total.

Esse processo é repetido iterativamente até que a função de custo seja minimizada, resultando em uma rede neural treinada capaz de realizar previsões precisas em novos dados. Essa propagação do erro é feita utilizando a regra da cadeia do cálculo diferencial, que permite calcular as derivadas parciais de funções compostas. O algoritmo de *backpropagation* pode ser resumido nos seguintes passos, apresentados no Algoritmo 2.

Entrada: Dados de treinamento (x, y) , taxa de aprendizado η , pesos W e vieses b

Saída: Pesos W e vieses b atualizados

Repita

Passo para frente: propagar x pela rede (camada a camada) e obter $\hat{y}$;

Cálculo do Erro: calcular o erro comparando $\hat{y}$ com y usando uma função de custo ;

Passo para trás: retropropagar o erro e calcular os gradientes $\nabla_W C$ e $\nabla_b C$ (regra da cadeia);

Atualização: atualizar pesos e vieses (descida do gradiente);

$W \leftarrow W - \eta \nabla_W C$;

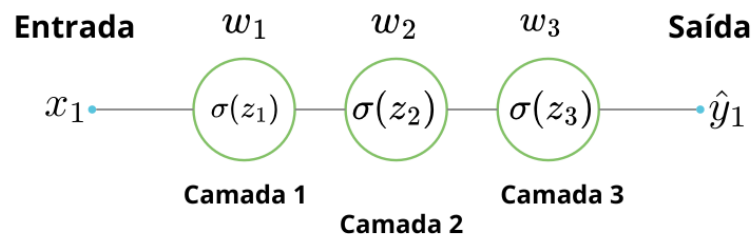
$b \leftarrow b - \eta \nabla_b C$;

até critério de parada (ex.: n^o de épocas ou convergência);

Algoritmo 2: Etapas do treinamento via *backpropagation*.

Na Figura 1.9 ilustra uma rede neural simples e a equação (1.10) mostra o cálculo do gradiente para ajustar o peso da primeira camada (w_1) utilizando a regra da cadeia,

Figura 1.9: Ilustração do processo de *backpropagation* em uma rede neural.



Fonte: Elaboração do Autor.

$$\nabla_w C = \frac{\partial C(w, b)}{\partial w_1} = \frac{C(w, b)}{\sigma(z_3)} \cdot \frac{\sigma(z_3)}{z_3} \cdot \frac{z_3}{\sigma(z_2)} \cdot \frac{\sigma(z_2)}{z_2} \cdot \frac{z_2}{\sigma(z_1)} \cdot \frac{\sigma(z_1)}{z_1} \cdot \frac{z_1}{w_1}, \quad (1.10)$$

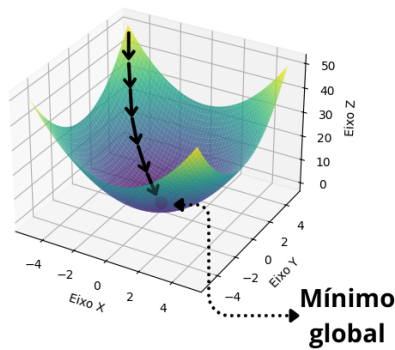
em que $z_1 = w_1 \cdot x + b_1$, $z_2 = \sigma(z_1) \cdot w_2 + b_3$ e $z_3 = \sigma(z_2) \cdot w_3 + b_3$ são as saídas das camadas ocultas e $C(w, b)$ é a função de custo do erro quadrático médio, ou seja, $C(w, b) = \frac{(\hat{y} - \sigma(z_3))^2}{n}$, n sendo o número de amostras no conjunto de dados.

1.2.3 Gradiente Descendente Estocástico

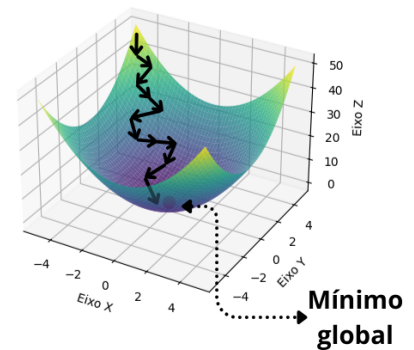
O Gradiente Descendente Estocástico (SGD - Stochastic Gradient Descent) (RUDER, 2016) é uma variação do método de descida do gradiente tradicional, amplamente utilizado no treinamento de redes neurais e outros modelos de aprendizado de máquina. A principal diferença entre o SGD e o método de descida do gradiente padrão é a forma como os gradientes são calculados e atualizados durante o treinamento. No método de descida do gradiente tradicional, os gradientes são calculados usando todo o conjunto de dados de treinamento, ou seja, os pesos são atualizados uma vez em cada época de treinamento, o que pode ser computacionalmente caro e demorado, especialmente para grandes conjuntos de dados. Em contraste, o SGD calcula os gradientes usando apenas uma amostra aleatória (ou um pequeno lote) do conjunto de dados em cada iteração, ou seja, caso o conjunto de dados tenha um tamanho de 1000 e o tamanho do lote seja 10, o SGD atualizará os pesos 100 vezes por época. Isso torna o processo de atualização dos pesos mais rápido e eficiente, permitindo que o modelo aprenda mais rapidamente. Apesar da Descida do Gradiente padrão fornecer uma direção mais precisa para o mínimo global da função de custo, o SGD é capaz de chegar próximo do mínimo de forma mais rápida e eficiente.

A Figura 1.10c ilustra como funciona o Gradiente Descendente Estocástico, em que cada seta preta indica uma atualização dos pesos do modelo. A Figura 1.10a ilustra o caminho percorrido pelo método de descida do gradiente tradicional, enquanto a Figura 1.10b ilustra o caminho percorrido pelo SGD.

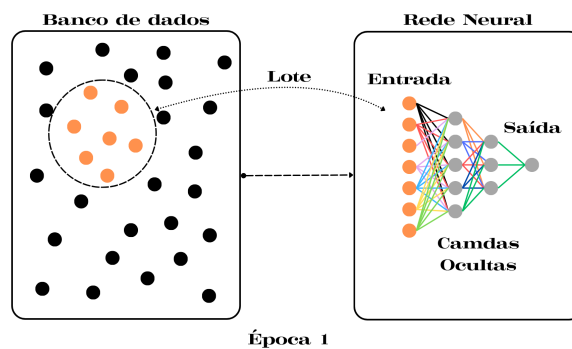
Figura 1.10: Comparação entre o método de descida do gradiente tradicional e o Gradiente Descendente Estocástico.



(a) Descida do Gradiente Tradicional.



(b) Gradiente Descendente Estocástico (SGD).



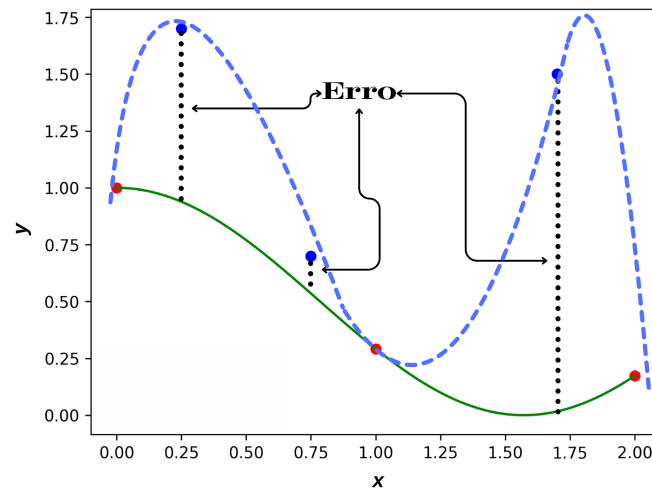
(c) Separando em lotes o banco de dados para o SGD.

Fonte: Elaboração do Autor.

1.2.4 *Overfitting*

O *Overfitting* (BOOK, 2025b) (do inglês “Sobreajuste” ou “Ajuste Excessivo”) é um problema comum em aprendizado de máquina, onde um modelo aprende os detalhes e o ruído dos dados de treinamento a tal ponto que este se torna incapaz de generalizar para novos dados. Isso significa que o modelo tem um desempenho excelente nos dados de treinamento, mas falha ao fazer previsões precisas em dados não vistos anteriormente.

O *overfitting* ocorre quando o modelo é muito complexo em relação à quantidade e qualidade dos dados disponíveis para treinamento. Modelos com muitos parâmetros, como redes neurais profundas, são particularmente suscetíveis ao *overfitting*, especialmente quando o conjunto de dados de treinamento é pequeno ou contém ruído significativo. Na Figura 1.11 é apresentada uma ilustração do *overfitting*, em que os pontos vermelhos são os dados de treinamento e os pontos azuis são os dados de teste, a curva verde representa um modelo que se ajusta perfeitamente aos dados de treinamento, mas falha em generalizar para novos dados, enquanto a curva azul seria um modelo ideal mais general.

Figura 1.11: Ilustração do *Overfitting*.

Fonte: Elaboração do Autor.

1.2.5 Hiperparâmetros

Um algoritmo de aprendizado pode ser visto como uma função que recebe dados de treinamento como entrada e produz como saída uma função ou modelo (isto é um conjunto de funções). No entanto, na prática muitos algoritmos de aprendizado possuem parâmetros adicionais que não são aprendidos diretamente a partir dos dados de treinamento, mas que influenciam o processo de aprendizado. Esses parâmetros são chamados de hiperparâmetros e sua escolha adequada é crucial para o desempenho do modelo (BENGIO, 2012). Alguns exemplos comuns de hiperparâmetros incluem:

- **Taxa de Aprendizado:** Controla o tamanho dos passos dados durante a atualização dos pesos do modelo. Uma taxa de aprendizado muito alta pode levar a oscilações e impedir a convergência, enquanto uma taxa muito baixa pode resultar em um processo de treinamento lento. Valores típicos para a taxa de aprendizado variam entre 0,01 e 0,1, dependendo do problema e do algoritmo utilizado.
- **Número de Épocas:** Define quantas vezes o algoritmo de aprendizado passará por todo o conjunto de dados de treinamento. Um número muito alto pode levar ao *overfitting*, enquanto um número muito baixo pode resultar em *underfitting* (subajuste). Uma técnica comum para determinar o número ideal de épocas é o uso do *Early Stopping* (do inglês Parada Antecipada), que monitora o desempenho do modelo em um conjunto de validação e interrompe o treinamento quando o desempenho começa a piorar.
- **Tamanho do Lote:** Refere-se ao número de amostras de dados usadas para calcular o gradiente em cada iteração do treinamento. Tamanhos de lote menores podem levar a atualizações mais ruidosas, enquanto tamanhos maiores podem resultar em um treinamento mais estável, embora mais lento. Os valores escolhidos geralmente variam entre 1 e 32, sendo que valores acima de 10 tiram proveito da Unidade Central de Processamento

(do inglês *Central Processing Unit* - CPU) e da Unidade de Processamento Gráfico (do inglês *Graphics Processing Unit* - GPU), que oferece ganho de velocidade dos produtos matriz-matriz em relação aos produtos matriz-vetor.

- **Arquitetura do Modelo:** Inclui o número de camadas, o número de unidades em cada camada e o tipo de funções de ativação utilizadas. Modelos mais complexos podem capturar padrões mais sofisticados, mas também são mais propensos ao *overfitting*.

1.2.6 Regularização

A regularização (BOOK, 2025c) é uma técnica utilizada para prevenir o *overfitting* em modelos de aprendizado de máquina, adicionando uma penalidade à função de custo que o modelo tenta minimizar durante o treinamento. Essa penalidade incentiva o modelo a manter os pesos dos parâmetros pequenos, o que ajuda a evitar que ele se ajuste excessivamente aos dados de treinamento. Existem várias técnicas de regularização, sendo as mais comuns:

- **Regularização L1 (Norma 1):** Adiciona uma penalidade proporcional à soma dos valores absolutos dos pesos dos parâmetros à função de custo. Isso pode levar a soluções esparsas, onde alguns pesos são reduzidos a zero, efetivamente removendo certas características do modelo. A equação da função de custo com regularização L1 é dada por:

$$C(w, b) = C_0(w, b) + \frac{\lambda}{n} \sum_i |w_i|, \quad (1.11)$$

em que $C_0(w, b)$ é a função de custo original, λ é o parâmetro de regularização que controla a força da penalidade, w_i são os pesos dos parâmetros do modelo e n é o número de amostras no conjunto de dados.

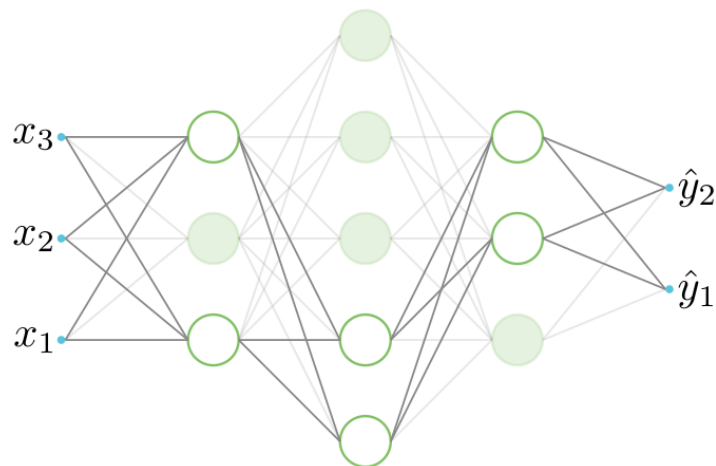
- **Regularização L2 (Norma 2):** Adiciona uma penalidade proporcional à soma dos quadrados dos pesos dos parâmetros à função de custo. Isso incentiva o modelo a distribuir os pesos de forma mais uniforme, evitando que alguns pesos se tornem muito grandes. A equação da função de custo com regularização L2 é dada por:

$$C(w, b) = C_0(w, b) + \frac{\lambda}{n} \sum_i w_i^2, \quad (1.12)$$

em que os termos são definidos da mesma forma que na regularização L1.

- **Dropout:** Durante o treinamento, essa técnica desativa aleatoriamente uma fração das unidades em cada camada da rede neural para cada época. Isso força a rede a aprender representações mais robustas e reduz a dependência de unidades específicas, ajudando a prevenir o *overfitting*. Em termos gerais, ao aplicar o *dropout* estamos treinando uma coleção de sub-redes dentro da rede original, o que pode promover a generalização do modelo.

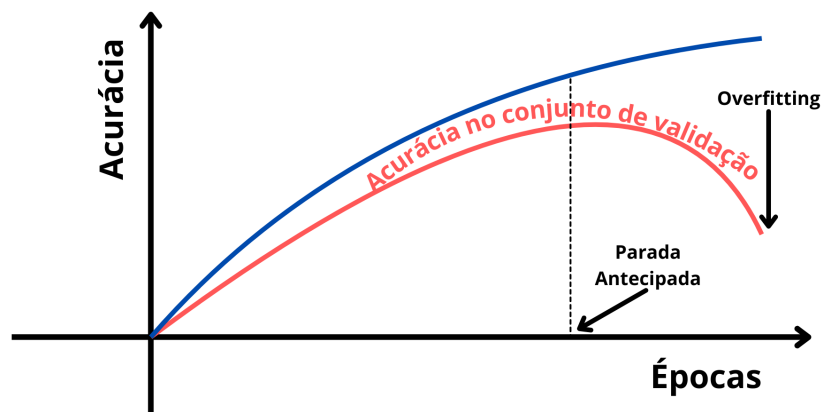
Figura 1.12: Ilustração do *Dropout* removendo 60% das unidades aleatoriamente durante uma época de treinamento.



Fonte: Elaboração do Autor.

- *Early Stopping* (Parada Antecipada): Monitora o desempenho do modelo em um conjunto de validação durante o treinamento e interrompe o processo quando o desempenho começa a piorar. Isso evita que o modelo continue a se ajustar aos dados de treinamento além do ponto ideal. Como ilustrado na Figura 1.13, o ponto de parada ideal é onde a acurácia na validação começa a diminuir, indicando o início do *overfitting*.

Figura 1.13: Ilustração do *Early Stopping*.



Fonte: Adaptado de (BOOK, 2025c).

1.2.7 Momentum

O *Momentum* (Momento) (SUTSKEVER, 2013) é uma técnica utilizada para acelerar o processo de treinamento de redes neurais, especialmente em situações onde a função de custo apresenta regiões íngremes ou planas. A ideia principal do *momentum* é acumular uma média

ponderada dos gradientes anteriores para suavizar as atualizações dos pesos, permitindo que o modelo mantenha uma direção consistente durante o treinamento. Isso ajuda a evitar oscilações e acelera a convergência para o mínimo da função de custo. Dada uma função de custo $C(w, b)$ a ser minimizada, o *momentum* clássico é definido por:

$$\begin{cases} v_{t+1} = \beta v_t - 1 + \epsilon \nabla C(w_t, b_t) \\ \theta_{t+1} = \theta_t + v_{t+1}, \end{cases} \quad (1.13)$$

em que $\epsilon > 0$ é a taxa de aprendizagem, $\beta \in [0, 1]$ é o coeficiente de *momentum*, v_t é a velocidade acumulada no tempo t , $\theta_t = (w_t, b_t)$ representa os parâmetros do modelo (pesos e vieses) no tempo t e $\nabla C(w_t, b_t)$ é o gradiente da função de custo em relação aos parâmetros do modelo. O coeficiente de *momentum* β controla a influência dos gradientes anteriores nas atualizações atuais. Valores típicos para β variam entre 0,9 e 0,99, com valores mais altos dando mais peso aos gradientes passados.

Colocar um subseção do ADAM, que é uma variação do Momentum.

1.2.8 *Vanishing Gradient*

O problema do *vanishing gradient* (gradiente que desaparece) ocorre durante o treinamento de redes neurais profundas, em que os gradientes calculados durante o processo de *backpropagation* tornam-se extremamente pequenos à medida que são propagados para as camadas mais próximas da entrada. Isso acontece devido à multiplicação repetida de pequenas derivadas associadas às funções de ativação e aos pesos das camadas. Como resultado, os pesos das camadas iniciais da rede recebem atualizações insignificantes, dificultando o aprendizado efetivo dessas camadas. Como neste trabalho serão utilizadas redes neurais profundas, é importante entender o impacto do problema do *vanishing gradient* e as técnicas para mitigá-lo.

Exemplo 1.2. Considere uma rede neural profunda com várias camadas ocultas, em que cada camada utiliza a função de ativação sigmoid logística, definida como:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (1.14)$$

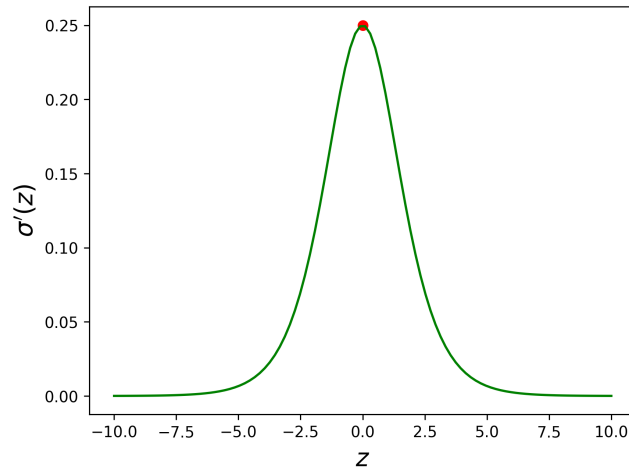
A derivada dessa função é dada por:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)). \quad (1.15)$$

Durante o processo de *backpropagation*, o gradiente da função de custo em relação aos pesos das camadas iniciais é calculado utilizando a regra da cadeia. Isso envolve a multiplicação das derivadas das funções de ativação de cada camada, como mostra a equação (1.10). Como a derivada da função sigmoid é sempre menor que 0,25 (o valor máximo ocorre em $z = 0$), a multiplicação repetida dessas pequenas derivadas resulta em gradientes que se tornam cada vez menores à medida que são propagados para trás através das camadas da rede. Por exemplo, se uma rede possui 10 camadas ocultas, o gradiente pode ser reduzido a um valor extrema-

mente pequeno, como 10^{-6} ou menor, tornando as atualizações dos pesos nas camadas iniciais praticamente insignificantes.

Figura 1.14: Gráfico da derivada da função sigmoid logística.



Fonte: Elaboração do Autor.

O problema do *vanishing gradient* é particularmente comum em redes neurais recorrentes (RNNs) e em redes *feedforward* profundas que utilizam funções de ativação como a sigmoid ou a tangente hiperbólica. Para contornar o problema do *vanishing gradient*, várias técnicas podem ser empregadas, incluindo:

- **Funções de Ativação Alternativas:** Funções de ativação como ReLU e suas variantes (Leaky ReLU, Parametric ReLU) têm derivadas constantes para valores positivos, o que ajuda a manter os gradientes maiores durante o *backpropagation*.
- **Batch Normalization:** Normaliza as ativações de cada camada durante o treinamento, o que pode ajudar a estabilizar os gradientes e acelerar o processo de aprendizado.

1.2.9 Dígitos Manuscritos

Para exemplificar o funcionamento de como uma rede neural faz o reconhecimento de imagens, considere a tarefa de classificação de dígitos manuscritos utilizando o conjunto de dados MNIST (LECUN, 1998). O conjunto de dados MNIST consiste em 70.000 imagens em escala de cinza de dígitos manuscritos (0 a 9), cada uma com uma resolução de 28×28 pixels, como mostra a Figura 1.15.

Cada imagem é representada como uma matriz de 28×28 , em que cada pixel tem um valor de intensidade entre 0 (preto) e 255 (branco). Para o reconhecimento dos dígitos utilizou-se uma rede *feedforward* e os métodos presentes em (1.2.2), (1.2.3), (1.2.6) e (1.2.8) para o treinamento e otimização dos pesos. Para compor a camada de entrada da rede neural, cada imagem é “achatada” em um vetor de 784 elementos ($28 \times 28 = 784$), onde cada elemento corresponde à intensidade de um pixel. O objetivo da rede neural *feedforward* com camadas

Figura 1.15: Exemplos de dígitos manuscritos do conjunto de dados MNIST.

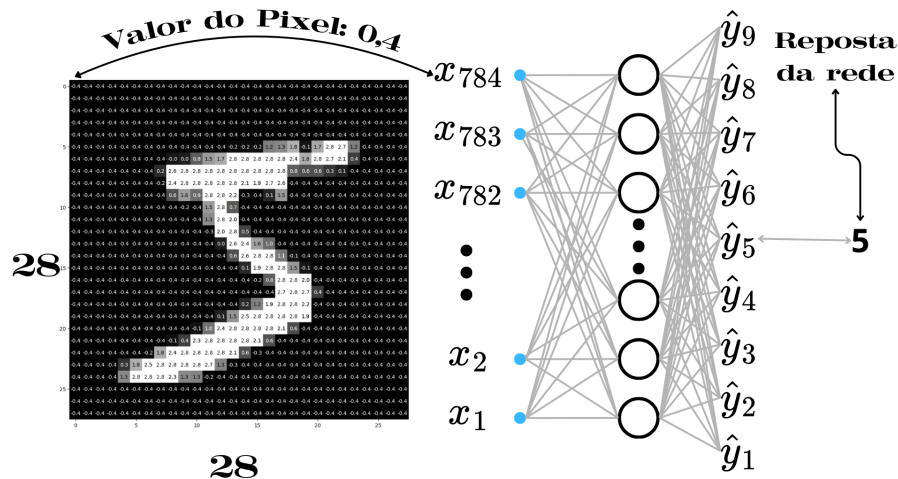


Fonte: Adaptado de (LECUN, 1998).

totalmente conectadas é aprender a classificar corretamente cada imagem em uma das 10 classes correspondentes aos dígitos de 0 a 9. Como a camada de saída da rede neural deve ter 10 unidades (um para cada classe), a arquitetura da rede pode ser definida como $[784, 30, 9]$, onde a camada de entrada tem 784 unidades, a camada oculta tem 30 unidades (valor escolhido aleatoriamente) e a camada de saída tem 9 unidades, como mostra a Figura 1.16.

Figura 1.16: Arquitetura da rede neural para classificação de dígitos manuscritos.

Inicialmente, é enviado o número de pixels da imagem (784) para a camada de entrada (unidade $x_{784} = 0,4$ intensidade do pixel), que é processada por uma camada oculta com 30 unidades, e a camada de saída tem 9 unidades, correspondendo às 10 classes de dígitos (0 a 9).



Fonte: Elaboração do Autor.

A função de ativação utilizada é a função sigmoid logística vista na equação (1.3), que é comumente usada em tarefas de classificação. A saída da rede é um vetor de 10 elementos, em que cada elemento representa a probabilidade da imagem pertencer a uma das classes

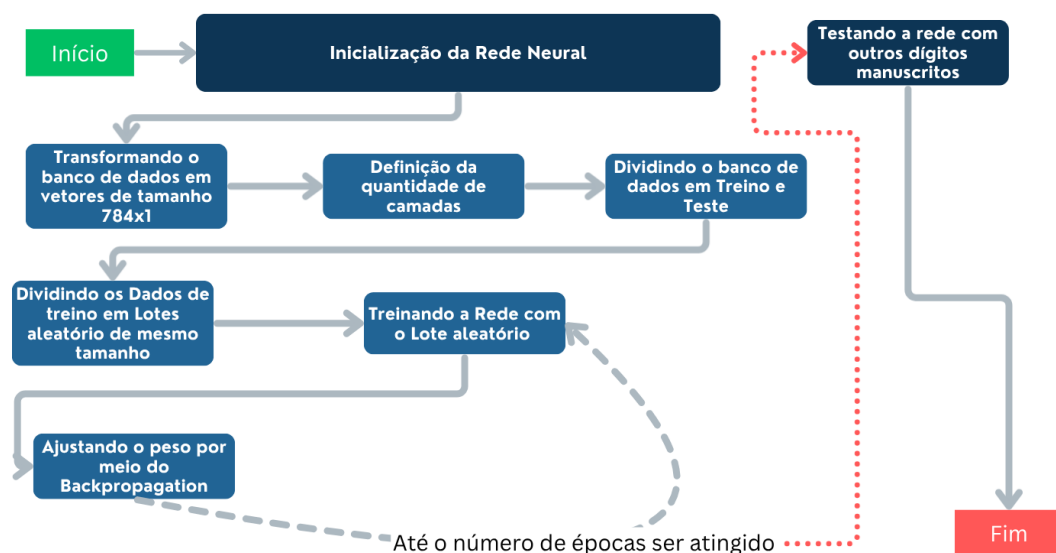
de dígitos. A classe com a maior probabilidade é escolhida como a previsão da rede para a imagem de entrada (para a Figura 1.16 a resposta da rede é 5, que foi a unidade com a maior probabilidade).]

Para o treinamento da rede é utilizado o algoritmo de *backpropagation* (Algoritmo 2) e o Gradiente Descendente Estocástico, com uma taxa de aprendizagem de 0,1 e a normalização *L1* visto na equação (1.11). O treinamento é realizado para 30 épocas. O banco de dados é dividido em um conjunto de treinamento com 60.000 imagens e um conjunto de teste com 10.000 imagens, aleatoriamente. Após o treinamento, a rede neural é avaliada no conjunto de teste para medir sua acurácia, que é a proporção de previsões corretas em relação ao total de previsões feitas:

$$\text{Acurácia} = \frac{\text{Número de Previsões Corretas}}{\text{Número Total de Previsões}}. \quad (1.16)$$

A Figura 1.17 mostra o fluxo de dados durante o processo de treinamento e avaliação da rede neural para classificação de dígitos manuscritos.

Figura 1.17: Fluxo de dados durante o processo de treinamento e avaliação da rede neural.

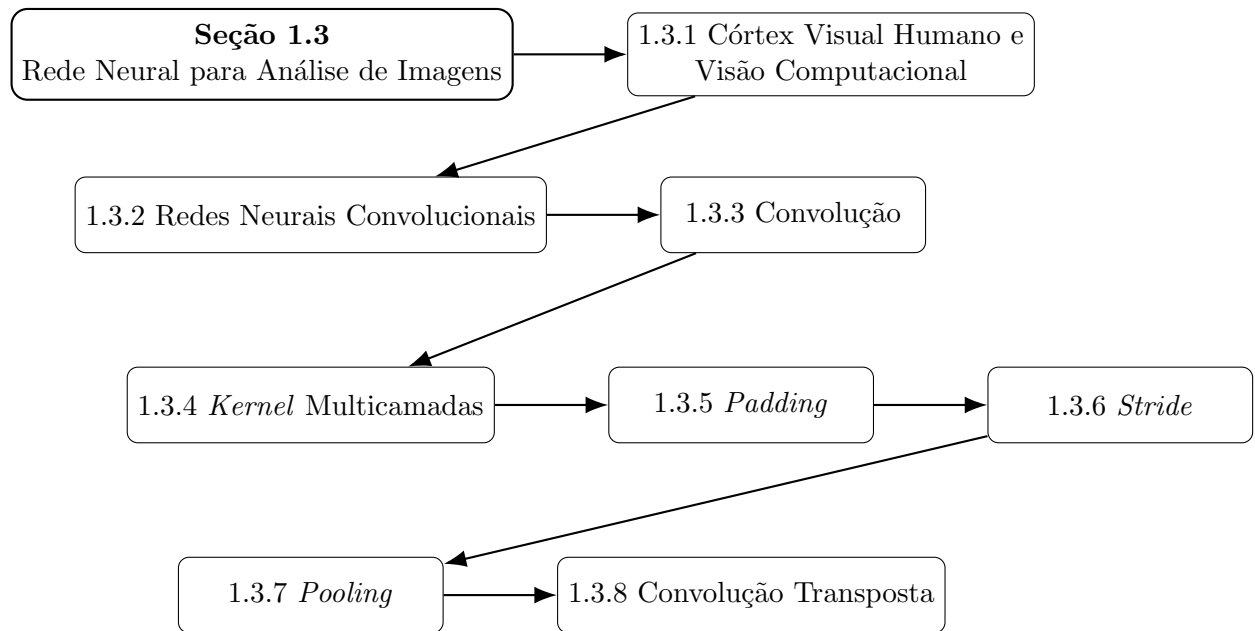


Fonte: Elaboração do Autor.

Com as configurações mencionadas, a rede neural alcançou uma acurácia de aproximadamente 95% no conjunto de teste, o que indica que esta foi capaz de aprender a classificar corretamente a maioria das imagens de dígitos manuscritos.

1.3 Rede Neural para Análise de Imagens

Figura 1.18: Organização da Seção 1.3.



Fonte: Elaboração do Autor.

1.3.1 Córtex Visual Humano e Visão Computacional

O córtex visual humano é a parte do cérebro responsável pelo processamento das informações visuais recebidas pelos olhos (STANFIELD, 2014). Este está localizado na parte posterior do cérebro, na região occipital. O córtex visual é organizado em várias áreas, cada uma especializada em processar diferentes aspectos da visão, como cor, forma, movimento e profundidade. Essas áreas trabalham em conjunto para interpretar e compreender o mundo visual ao nosso redor. A estrutura do córtex visual é altamente complexa, com bilhões de unidades interconectadas que formam redes neurais intrincadas. Essas redes são capazes de aprender e adaptar-se com base nas experiências visuais, permitindo-nos reconhecer objetos, rostos e padrões visuais de maneira eficiente. Uma característica importante do córtex visual é a presença de campos receptivos, que são regiões específicas do campo visual onde a estimulação de uma unidade resulta em uma resposta. Esses campos receptivos são organizados de forma hierárquica, com unidades em níveis mais baixos respondendo a características simples, como bordas e texturas, enquanto unidades em níveis mais altos respondem a características mais complexas, como formas e objetos completos. É por meio desse mecanismo e também pela capacidade de aprendizagem que o ser humano é capaz de reconhecer e interpretar o mundo visual ao seu redor.

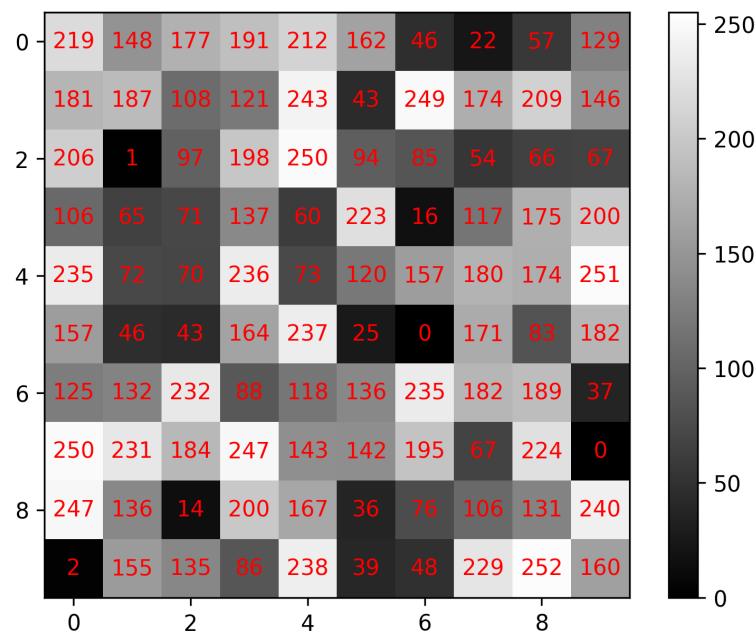
Traduzir a visão para uma representação digital é um processo extremamente complexo. Em parte, porque é um problema inverso, no qual busca-se recuperar algumas variáveis desconhecidas a partir de informações insuficientes para especificar completamente a solução. Portanto, deve-se recorrer a modelos baseados na física, modelos probabilísticos ou aprendizado

de máquina com grandes conjuntos de exemplos para resolver ambiguidades entre possíveis soluções. Entretanto, modelar o mundo visual em toda a sua rica complexidade é muito mais difícil do que, por exemplo, modelar o trato vocal que produz sons da fala (SZELISKI, 2010).

É surpreendente que humanos e animais façam isso com tanta facilidade, enquanto algoritmos de visão computacional são tão propensos a erros. Pessoas que nunca trabalharam na área frequentemente subestimam a dificuldade do problema. Essa percepção equivocada de que a visão deveria ser fácil remonta aos primeiros dias da inteligência artificial, quando se acreditava inicialmente que as partes cognitivas (prova lógica e planejamento) da inteligência eram intrinsecamente mais difíceis do que os componentes perceptivos (MÜLLER, 2008)².

Uma imagem virtual é uma representação visual de um objeto, cena ou conceito, composta por uma matriz de pixels que contém informações sobre cor e intensidade. Cada pixel é uma célula na matriz da imagem que possui um valor específico. As imagens podem ser em escala de cinza, em que cada pixel tem um valor de intensidade entre 0 (preto) e 255 (branco), como mostra a Figura 1.19, ou em cores, em que cada pixel é representado por uma combinação de valores para as cores primárias (vermelho, verde e azul - RGB), uma imagem colorida pode ser interpretada como a sobreposição de três matrizes, cada matriz representando uma das cores primárias.

Figura 1.19: Exemplo de uma imagem/matriz 10×10 em escala de cinza, em que cada pixel tem um valor de intensidade entre 0 (preto) e 255 (branco).



Fonte: Elaboração do Autor.

²(MÜLLER, 2008) cita (CREVIER, 1993) como a fonte original. Entretanto o artigo original foi escrito por Seymour Papert (1966) e envolveu uma turma de estudantes.

1.3.2 Redes Neurais Convolucionais

As redes neurais convolucionais (RNCs ou do inglês *Convolutional Neural Networks* - CNNs) são uma classe especial de redes neurais projetadas para processar dados com uma estrutura de grade e que tenham uma relação de vizinhança entre os elementos, como imagens (TREVISAN, 2021), foram baseadas do Cortex Visual Humano (1.3.1). Estas são amplamente utilizadas em tarefas de visão computacional, como reconhecimento de objetos, detecção de faces e segmentação³ de imagens. As RNCs são inspiradas na organização do córtex visual, onde diferentes regiões do cérebro são responsáveis por processar diferentes partes do campo visual.

1.3.3 Convolução

Exemplo 1.3. *Supondo que na tentativa de localizar uma nave com um sensor a laser (GODFELLOW; BENGIO; COURVILLE, 2016), este sensor retorna uma única saída $x(t)$, a posição da nave no tempo t . $x(t)$ e t são valores reais, ou seja, é possível adquirir diferentes leituras do sensor em qualquer instante de tempo. Supondo que o sensor seja ruidoso, para obter pouco ruído da estimativa da posição da nave é possível fazer várias medições e calcular a média destas, entretanto, medições mais recentes são mais relevantes, então, pode-se fazer uma média ponderada que dê pesos às medições mais recentes. Seja esta média a função $w(a)$, em que a é o tempo em que a medição foi obtida. Ao aplicar $w(a)$ à $x(t)$, para todo t , é obtida a função s , que é a estimativa suavizada da posição da nave, ou seja:*

$$s(t) = \int x(a)w(t-a)da. \quad (1.17)$$

Essa operação é chamada de convolução (SOVIERZOSKI, 2010). A operação de convolução é tipicamente denotada com um asterístico:

$$s(t) = (x * w)(t).$$

No exemplo 1.3, é preciso que w seja uma função densidade de probabilidade válida para que a saída seja uma média ponderada e w precisa ser igual a zero para $a < 0$. Estas restrições são particulares do exemplo, em geral, convolução é definida para quaisquer funções e integrais.

Na terminologia da rede convolucional, o primeiro termo (x) é frequentemente chamado de *input* (entrada), o segundo termo (w) é chamado de *kernel* (núcleo) ou filtro, e o resultado (s) é chamado de *feature map* (mapa de características).

A operação de convolução é comumente aplicada a dados provenientes de domínios discretos (dados de computador). Se (x) e (w) são definidas somente para valores inteiros de t , a convolução discreta é definida por:

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)w(t-a), \quad (1.18)$$

³Posteriormente será explicado por que o termo segmentação pode ser trocado por máscara.

em muitos materiais, a convolução discreta pode ser representada por:

$$s[t] = (x * w)[t] = \sum_{a=-\infty}^{\infty} x[a]w[t-a]$$

Exemplo 1.4. Seja $f(t) = e^{-t}u(t)$ e $g(t) = u(t)$, onde $u(t)$ é a função limiar equação (1.2). A convolução $(f * g)(t)$ é dada por:

$$(f * g)(t) = \int_{-\infty}^{\infty} e^{-a}u(a)u(t-a)da.$$

Para $t < 0$, a função limiar $u(t-a)$ é zero para todos os valores de a , o que resulta em $(f * g)(t) = 0$. Para $t \geq 0$, a função limiar $u(t-a)$ é igual a 1 para $a \leq t$ e zero para $a > t$. Portanto, a convolução pode ser reescrita como:

$$(f * g)(t) = \int_0^t e^{-a}da.$$

Calculando a integral, obtém-se:

$$(f * g)(t) = [-e^{-a}]_0^t = 1 - e^{-t}.$$

Portanto, a convolução entre as funções $f(t)$ e $g(t)$ é dada por:

$$(f * g)(t) = \begin{cases} 0, & t < 0 \\ 1 - e^{-t}, & t \geq 0 \end{cases} \quad (1.19)$$

Intuitivamente, a função g é invertida e deslocada por t e depois multiplicada ponto a ponto com a função f , e o resultado é integrado sobre todo o domínio contínuo. Para domínios discretos, o Exemplo 1.5 mostra o processo de convolução entre duas sequências finitas.

Exemplo 1.5. Seja $x[n] = \{1, 2, 3\}$ e $w[n] = \{0, 1\}$, em que $x[0] = 1$, $x[1] = 2$, $x[2] = 3$, $w[0] = 0$ e $w[1] = 1$ e $w[n] = x[n] = 0$ para outros valores de n . A convolução $(x * w)[n]$ é dada por:

$$s[n] = (x * w)[n] = \sum_{a=0}^3 x[a]w[n-a].$$

Calculando os valores para cada n , obtém-se:

$$\begin{aligned} (n=0) \quad (x * w)[0] &= x[0]w[0] + x[1]w[-1] + x[2]w[-2] = 1 \cdot 0 + 2 \cdot 0 + 3 \cdot 0 = 0 \\ (n=1) \quad (x * w)[1] &= x[0]w[1] + x[1]w[0] + x[2]w[-1] = 1 \cdot 1 + 2 \cdot 0 + 3 \cdot 0 = 1 \\ (n=2) \quad (x * w)[2] &= x[0]w[2] + x[1]w[1] + x[2]w[0] = 1 \cdot 0 + 2 \cdot 1 + 3 \cdot 0 = 2 \\ (n=3) \quad (x * w)[3] &= x[0]w[3] + x[1]w[2] + x[2]w[1] = 1 \cdot 0 + 2 \cdot 0 + 3 \cdot 1 = 3 \end{aligned}$$

Portanto, a convolução entre as sequências $x[n]$ e $w[n]$ é dada por:

$$(x * w)[n] = \{0, 1, 2, 3\}. \quad (1.20)$$

Proposição 1.1. *O tamanho do vetor resultante da convolução entre dois vetores de comprimento M e N é dado por $M + N - 1$.*

Demonstração. Seja $x[n] \neq 0$ para $0 \leq n \leq M - 1$ e $w[n] \neq 0$ para $0 \leq n \leq N - 1$. Duas sequências finitas de comprimentos M e N respectivamente. Então, a convolução $(x * w)[n]$ é dada por:

$$y[n] = (x * w)[n] = \sum_{a=-\infty}^{\infty} x[a]w[n-a],$$

para que a soma seja diferente de zero, é necessário que $x[a] \neq 0$ e $w[n-a] \neq 0$, ou seja:

$$0 \leq a \leq M - 1 \quad \text{e} \quad 0 \leq n - a \leq N - 1.$$

Somando a na segunda desigualdade é obtido:

$$a \leq n \leq a + N - 1$$

Como a pode variar entre 0 e $M - 1$, quando $a = 0$, então $n \geq 0$ e quando $a = M - 1$, então $n \leq M - 1 + N - 1 = M + N - 2$. Portanto:

$$0 \leq n \leq M + N - 2,$$

o que implica que o vetor resultante $(y[n])$ tem comprimento $M + N - 1$ □

No Exemplo 1.5, a convolução entre os vetores $x[n]$ e $w[n]$ resultou em um vetor de comprimento 4, que é igual a $3 + 2 - 1 = 4$. Para voltar ao comprimento original do vetor $x[n]$, pode-se fazer um truncamento do vetor resultante, ou seja, remover a mesma quantidade de elementos do início e do final do vetor resultante, o que é conhecido como convolução “*same*” (ou mesma em português). No exemplo, removendo o primeiro e o último elemento do vetor resultante, obtém-se a convolução *same*:

$$(x * w)_{\text{same}}[n] = \{1, 2\}.$$

Pode-se pensar na operação de convolução discreta como o processo de multiplicação e soma, em que o kernel w vai sendo “passado” sobre o vetor x ponto a ponto, como mostra a Figura 1.20.

No contexto de redes neurais convolucionais, a convolução é aplicada a dados discretos, como imagens representadas por matrizes de pixels. Nesse caso, o sinal x é uma matriz bidimensional que representa a imagem de entrada e o *kernel* w é uma matriz menor que atua como um filtro para extrair características específicas da imagem. A operação de convolução em dados discretos é realizada deslizando o *kernel* sobre a imagem e calculando a soma ponderada dos valores dos pixels cobertos pelo *kernel*. A saída da convolução é uma nova matriz, chamada de Mapa de

Figura 1.20: Ilustração da operação de convolução discreta. O kernel w é deslizado sobre o vetor x , e em cada posição, a soma ponderada dos valores de x cobertos pelo kernel é calculada para gerar o valor correspondente na saída.

$$\begin{array}{c}
 \begin{array}{ccc}
 \begin{array}{c} (1, 2, 3) \\ | \\ (1, 0) \end{array} \} = 1 \cdot 0 = 0 & \longrightarrow & \begin{array}{c} (1, 2, 3) \\ | \\ (1, 0) \end{array} \} = 1 \cdot 1 + 2 \cdot 0 = 1 \\
 & & \searrow \\
 \begin{array}{ccc}
 \begin{array}{c} (1, 2, 3) \\ | \\ (1, 0) \end{array} \} = 2 \cdot 1 + 3 \cdot 0 = 2 & \longrightarrow & \begin{array}{c} (1, 2, 3) \\ | \\ (1, 0) \end{array} \} = 3 \cdot 1 = 3
 \end{array}
 \end{array}$$

Vetor resultante: $(0, 1, 2, 3)$

Fonte: Elaboração do Autor.

Características, que destaca as características extraídas pela aplicação do *kernel*. No caso de uma convolução bidimensional discreta, a operação é definida como:

$$S[i, j] = (X * W)[i, j] = \sum_m \sum_n X[m, n] W[i - m, j - n], \quad (1.21)$$

convolução é uma operação comutativa, ou seja:

$$S[i, j] = (X * W)[i, j] = \sum_m \sum_n X[i - m, j - n] W[m, n],$$

considerando que X é a matriz da imagem de entrada, W é a matriz do kernel e S é o mapa de características resultante.

Exemplo 1.6. Considere a convolução de duas matrizes, em que a matriz de entrada

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

é convoluída com o kernel

$$W = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

para produzir a saída S . Então, como nos exemplos anteriores, é necessário inverter o kernel W e depois deslizar o kernel sobre a matriz de entrada X . Neste exemplo, a convolução é same.

$$W \rightarrow \hat{W} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix},$$

$\hat{W}$ foi obtido fazendo uma rotação de 180 graus do kernel original W , isso acontece pois na definição, para $i = 0$ e $j = 0$, temos $\hat{W}[i, j] = W[-m, -n]$. Para facilitar o cálculo da convolução, pode-se separar X com blocos de 2×2 (tamanho do kernel) e calcular a convolução para cada bloco. Para o bloco superior esquerdo de X até o bloco inferior direito, temos:

$$\begin{aligned} S(0, 0) &= \begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix} \odot \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} = (1)(-1) + (2)(0) + (4)(0) + (5)(1) = 4, \\ S(0, 1) &= \begin{bmatrix} 2 & 3 \\ 5 & 6 \end{bmatrix} \odot \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} = (2)(-1) + (3)(0) + (5)(0) + (6)(1) = 4, \\ S(1, 0) &= \begin{bmatrix} 4 & 5 \\ 7 & 8 \end{bmatrix} \odot \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} = (4)(-1) + (5)(0) + (7)(0) + (8)(1) = 4, \\ S(1, 1) &= \begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix} \odot \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} = (5)(-1) + (6)(0) + (8)(0) + (9)(1) = 4, \end{aligned}$$

em que a operação $\odot$ significa a multiplicação de cada elemento correspondente das matrizes e a soma dos resultados. Portanto, a saída da convolução é dada por:

$$S = \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}. \quad (1.22)$$

Observa-se que a saída da convolução é uma matriz de 2×2 . O cálculo da dimensão da matriz de saída é feito da seguinte forma: $Largura_S = largura_X - largura_W + 1 = 3 - 2 + 1 = 2$ e $Altura_S = altura_X - altura_W + 1 = 3 - 2 + 1 = 2$.

Na Figura 1.21 é ilustrado um exemplo de convolução bidimensional, em que uma matriz de entrada X (matriz 7×7) é convoluída com um kernel W (matriz 2×2) para produzir a saída S (matriz 6×6). W é deslizado sobre a matriz de entrada, e em cada posição, a soma ponderada dos valores dos pixels cobertos pelo kernel é calculada e o resultado é aplicado em uma função de ativação (σ) para gerar o valor correspondente na matriz de saída, assim como no Exemplo 1.6. No caso da Figura 1.21, é utilizado a convolução válida, ou seja, a saída é gerada apenas para as posições onde o kernel pode ser completamente aplicado à matriz de entrada sem ultrapassar seus limites.

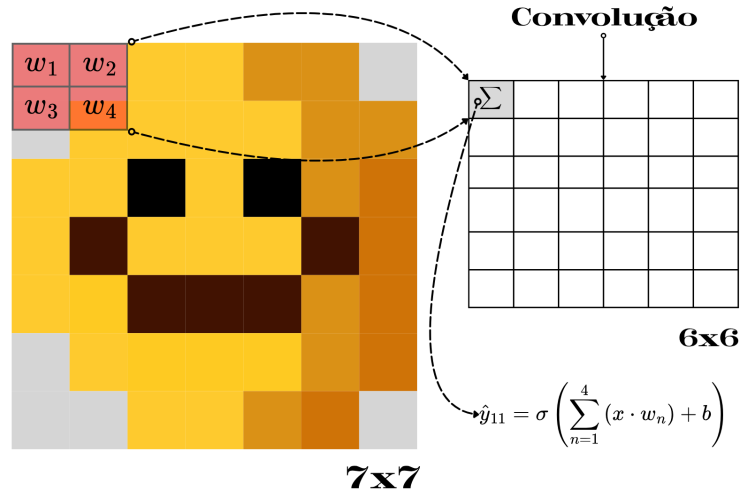
O cálculo da largura e da altura da imagem de saída pode ser feito utilizando as seguintes fórmulas:

$$w_{out} = w_{in} - w_{ker} + 1, \quad (1.23)$$

$$h_{out} = h_{in} - h_{ker} + 1, \quad (1.24)$$

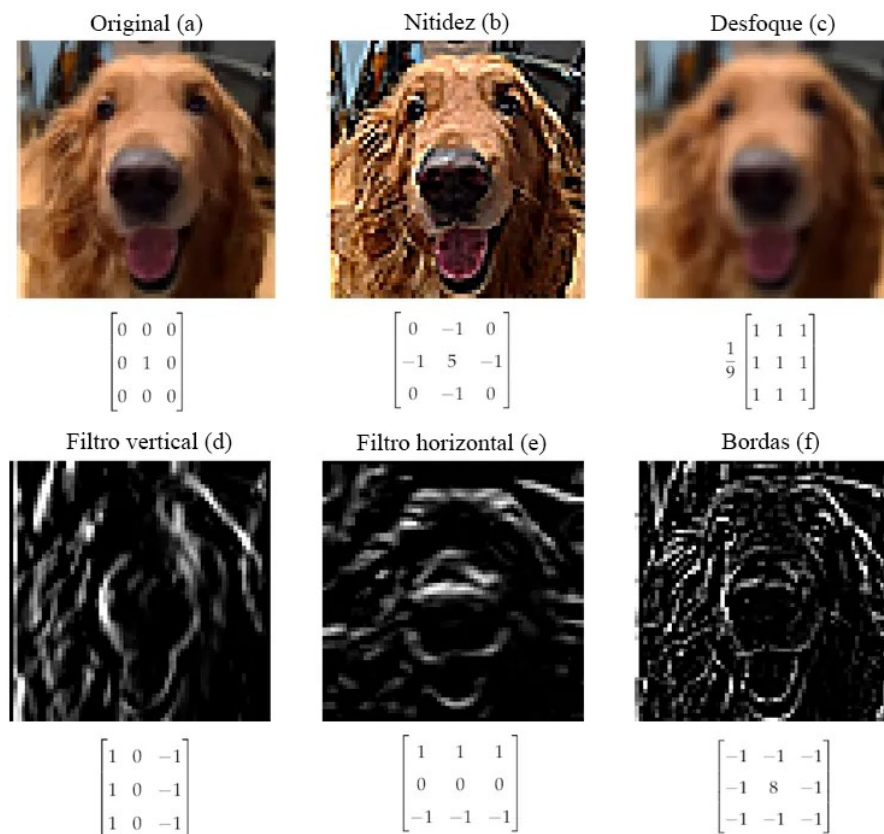
em que w_{out} e h_{out} são a largura e a altura da imagem de saída, w_{in} e h_{in} são a largura e a altura da imagem de entrada, e w_{ker} e h_{ker} são a largura e a altura do kernel, respectivamente. A Figura 1.22 mostra diferentes aplicações de kernels em uma imagem.

Figura 1.21: Exemplo de convolução bidimensional. A matriz de entrada X (matriz 7×7) é convoluída com o *kernel* W (matriz 2×2) para produzir a saída S (matriz 6×6). O *kernel* é “deslizado” sobre a matriz de entrada, formando sub-matrizes, e em cada posição, a soma ponderada dos valores dos pixels cobertos pelo *kernel* é calculada e o resultado é aplicado em uma função de ativação (σ) para gerar o valor correspondente ($\hat{y}_{11}$) na matriz de saída. b sendo o viés adicionado à soma ponderada.



Fonte: Elaboração do Autor.

Figura 1.22: Aplicações de diferentes *kernels* em uma imagem. A imagem original é representada em (a), juntamente com o *kernel* identidade, que não faz modificações. Em (b) a imagem passou por um *kernel* que aumenta a nitidez reforçando o pixel central e aumentando a diferença dele em relação à sua vizinhança. Em (c) foi usado um *kernel* de desfoque, que substitui um pixel pela média dele com seus vizinhos. A imagem em (d) é o resultado de aplicar na imagem em escala de cinza um *kernel* que reforça suas linhas verticais, enquanto (e) reforça suas linhas horizontais. O *kernel* em (f) reforça os contornos (bordas) da imagem.



Fonte: Adaptado de (TREVISAN, 2021).

Normalmente os *kernels* são muito menores do que as imagens (geralmente 3×3), e através da operação de convolução um *kernel* é utilizado para processar toda uma imagem, portanto, diz-se que os parâmetros são compartilhados (pesos e viés). Por conta dessa representação reduzida, cada camada da rede convolucional deve aprender uma quantidade menor de parâmetros. Em CNNs, normalmente considera-se a convolução como uma camada da rede, e cada camada normalmente possui múltiplos *kernels*, produzindo a mesma quantidade de mapas de atributos, cada um responsável por extrair diferentes características da imagem de entrada. Durante o treinamento da rede, os valores dos *kernels* são ajustados para otimizar a capacidade da rede de reconhecer padrões específicos nas imagens. Por exemplo, uma camada convolucional com 3 *kernels* que recebe como entrada uma imagem em escala de cinza de dimensões $(w_{in} \times h_{in} \times 1)$ produzirá uma saída com dimensões $(w_{out} \times h_{out} \times 3)$, onde w_{out} e h_{out} são calculados conforme as fórmulas apresentadas em (1.23) e (1.24). A saída não é mais uma imagem comum, mas sim um conjunto de três mapas de características, cada um representando a resposta da imagem de entrada a um dos três *kernels* aplicados. Encadeando os mapas de atributos, a rede pode aprender representações hierárquicas das características presentes nas imagens, desde bordas e texturas simples até formas e objetos mais complexos.

1.3.4 Correlação Cruzada

As redes neurais convolucionais geralmente não utilizam a definição tradicional de convolução (GOODFELLOW; BENGIO; COURVILLE, 2016), mas sim uma operação chamada correlação cruzada, definida como:

$$S[i, j] = (X * W)[i, j] = \sum_m \sum_n X[i + m, j + n] W[m, n], \quad (1.25)$$

em que X é a matriz da imagem de entrada, W é a matriz do *kernel* e S é o mapa de características resultante. Isso acontece porque no processo de treinamento os valores do *kernel* não são definidos manualmente, mas sim aprendidos por meio dos processos de otimização dos pesos. Por conta disso, a rede aprende a melhor configuração do *kernel* sem a necessidade de realizar a rotação de 180 graus, sendo exatamente o que a definição de correlação cruzada descreve, já que W não depende de parâmetros negativos.

1.3.5 Kernel Multicamadas

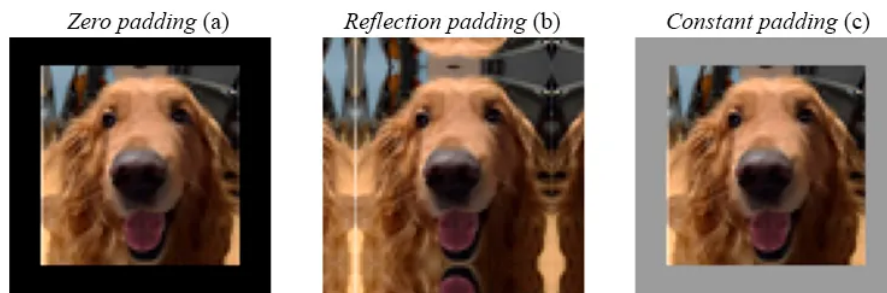
Em uma imagem com mais de um canal, como no caso de uma imagem VVA (vermelha, verde, azul) do inglês RGB (*Red, Green, Blue*), a convolução acontece com um *kernel* em cada canal da imagem, um conjunto de canais é chamado de filtro. Por exemplo, considere uma imagem RGB de dimensões $(w_{in} \times h_{in} \times 3)$ e um *kernel* de dimensões $(k_w \times k_h \times 3)$. A convolução é realizada separadamente em cada canal (vermelho, verde e azul) da imagem, utilizando o respectivo canal do *kernel*. Após a aplicação do *kernel* em cada canal, os resultados são somados ou é aplicada uma média para produzir um único mapa de características. Portanto, a saída da

convolução terá dimensões $(w_{out} \times h_{out} \times 1)$, onde w_{out} e h_{out} são calculados conforme as fórmulas apresentadas em (1.23) e (1.24).

1.3.6 *Padding*

Durante a aplicação da convolução, é comum que a dimensão espacial (largura e altura) da imagem de saída seja menor do que a da imagem de entrada. Isso ocorre porque o *kernel* não pode ser aplicado nas bordas da imagem sem ultrapassar seus limites. Para mitigar essa redução de dimensão, uma técnica chamada *padding* (O'SHEA; NASH, 2015) é frequentemente utilizada. O *padding* envolve adicionar uma borda de pixels ao redor da imagem original antes de aplicar a convolução. Esses pixels adicionais podem ser preenchidos com zeros (*zero-padding*) ou com valores replicados dos pixels mais próximos da borda da imagem original, (*reflection padding*). O uso de *padding* permite que o *kernel* seja aplicado em todas as regiões da imagem, incluindo as bordas, preservando assim as dimensões espaciais da imagem de entrada na saída. A Figura 1.23 ilustra a aplicação de diferentes tipos de *padding* em uma imagem.

Figura 1.23: Aplicação de *padding* em uma imagem. Em (a) a imagem original passou por *zero padding*. Em (b) a imagem original passou por *reflection padding*. A imagem em (c) passou por um *padding* constante, semelhante em (a), mas com valor diferente de zero.



Fonte: Adaptado de (TREVISAN, 2021).

O cálculo da largura e da altura da imagem de saída com *padding* pode ser ajustado conforme as seguintes fórmulas:

$$\begin{aligned} w_{out} &= w_{in} - w_{kernel} + 2w_{pad} + 1, \\ h_{out} &= h_{in} - h_{kernel} + 2h_{pad} + 1, \end{aligned} \tag{1.26}$$

em que w_{pad} e h_{pad} são a quantidade de pixels adicionados como *padding* na largura e na altura, respectivamente.

1.3.7 *Stride*

O *stride* (passo) é um parâmetro utilizado nas operações de convolução e *pooling* em redes neurais convolucionais que determina o número de pixels que o *kernel* de *pooling* se desloca ao longo da imagem de entrada. Em outras palavras, o *stride* define o quanto o *kernel* avança a cada aplicação da operação. Um *stride* de 1 significa que o *kernel* se mova um pixel por vez,

enquanto um *stride* maior que 1 faz com que o *kernel* pule pixels, resultando em uma redução mais rápida das dimensões espaciais da imagem. O uso do *stride* afeta diretamente o tamanho da saída gerada pela operação de convolução. O cálculo da largura e da altura da imagem de saída com *stride* pode ser ajustado conforme as seguintes fórmulas:

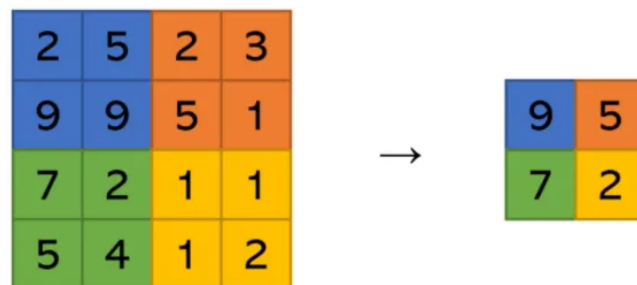
$$\begin{aligned} w_{out} &= \frac{w_{in} - w_{kernel} + 2w_{pad}}{s_w} + 1, \\ h_{out} &= \frac{h_{in} - h_{kernel} + 2h_{pad}}{s_h} + 1, \end{aligned} \quad (1.27)$$

em que s_w e s_h são os valores do *stride* na largura e na altura, respectivamente.

1.3.8 Pooling

O *pooling* (em tradução livre, *agrupamento*) é uma operação utilizada em redes neurais convolucionais para reduzir as dimensões espaciais (largura e altura) dos mapas de características, mantendo as informações mais relevantes. Essa redução ajuda a diminuir a complexidade computacional da rede. Existem diferentes tipos de operações de *pooling*, sendo as mais comuns o *Max Pooling* e o *Average Pooling* (Agrupamento Médio, técnica que leva em consideração a média dos valores dos pixels em cada bloco coberto pelo *kernel* de *pooling*). A técnica de *pooling* substitui agrupamentos de pixels por um valor que os represente. A Figura 1.24 ilustra o funcionamento do *max pooling* de tamanho 2, em que os pixels são agrupados em quadrados de tamanho 2x2 e são substituídos pelo maior valor de intensidade de pixel do grupo. Em CNN's o *stride* do *pooling* é geralmente igual ao tamanho do *kernel* de *pooling*, o que resulta em uma redução de dimensão espacial pela metade a cada aplicação de *pooling*. Por exemplo, um mapa de características de dimensões 8×8 aplicado a um *max pooling* com *kernel* de tamanho 2×2 e *stride* de 2 resultará em um mapa de características reduzido para dimensões 4×4 .

Figura 1.24: Exemplo de Max Pooling com tamanho de 2x2. A imagem original é reduzida para a imagem menor após a aplicação do Max Pooling. Cada bloco de 2x2 pixels na imagem original é substituído pelo valor máximo dentro desse bloco na imagem reduzida.



Fonte: Adaptado de (TREVISAN, 2021).

O *max pooling* pode ser implementado utilizando o Algoritmo 3, em que X é a matriz de entrada, k é o tamanho do *kernel* de *pooling* e Y é a matriz de saída resultante do *pooling*. O algoritmo percorre a matriz de entrada em blocos de tamanho $k \times k$, calculando o valor máximo dentro de cada bloco e atribuindo esse valor à posição correspondente na matriz de saída Y . O

resultado é uma matriz reduzida que mantém as características mais importantes da imagem original.

Entrada: X (matriz de entrada), k (tamanho do *kernel* de *pooling*)

Saída: Y (matriz de saída)

$w_{out} \leftarrow \lfloor w_{in}/k \rfloor$;

$h_{out} \leftarrow \lfloor h_{in}/k \rfloor$;

$Y \leftarrow$ matriz de zeros de tamanho $w_{out} \times h_{out}$;

Para $i \leftarrow 0$ **Até** $w_{out} - 1$ **Faça**

Para $j \leftarrow 0$ **Até** $h_{out} - 1$ **Faça**

$Y[i][j] \leftarrow \max(X[i * k : (i + 1) * k][j * k : (j + 1) * k]);$

Fim

Fim

Algoritmo 3: Max *pooling*

O cálculo da largura e da altura da imagem de saída após a aplicação do *pooling*, quando o *stride* é igual ao tamanho do *kernel*, pode ser realizado conforme as seguintes fórmulas:

$$\begin{aligned} w_{out} &= \lfloor \frac{w_{in}}{k} \rfloor, \\ h_{out} &= \lfloor \frac{h_{in}}{k} \rfloor, \end{aligned} \tag{1.28}$$

em que k é o tamanho do *kernel* de *pooling*, e $\lfloor \cdot \rfloor$ denota a função piso, que arredonda para baixo o resultado da divisão. Como consequência imediata da aplicação do *pooling*, a informação fica mais condensada, o que pode levar a uma perda de detalhes finos na imagem. No entanto, essa perda é compensada pela capacidade de aumentar a quantidade dos mapas de característica da imagem, melhorando sua capacidade de generalização em tarefas de reconhecimento de padrões.

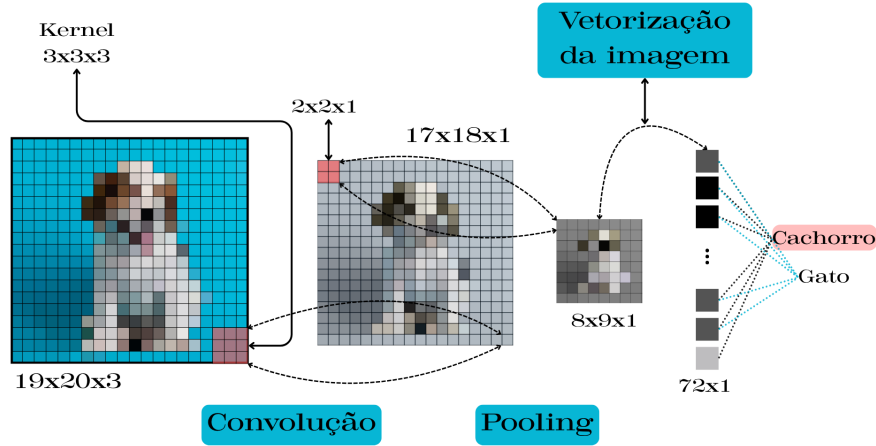
Exemplo 1.7. A Figura 1.25 apresenta uma arquitetura típica de uma Rede Neural Convolutiva (CNN), com uma camada de convolução e uma camada de *pooling*. Como a imagem tem três canais (RGB) de dimensões $19 \times 20 \times 3$ o *kernel* de convolução também tem três canais ($3 \times 3 \times 3$), a aplicação da convolução resulta em um mapa de características com um canal ($17 \times 18 \times 1$). O mapa de características é então processado por uma camada de *pooling* ($2 \times 2 \times 1$), que reduz suas dimensões espaciais ($8 \times 9 \times 1$). Após essa etapa a rede transforma o mapa de características em um vetor unidimensional (72×1), que é processado por camadas totalmente conectadas, visto na Seção 1.2 para realizar a classificação do objeto em cachorro ou gato, com a rede escolhendo a classe com maior probabilidade como resultado final (cachorro).

1.3.9 Convolução Transposta

A convolução transposta⁴, também conhecida como deconvolução, é uma operação utilizada em redes neurais convolucionais para aumentar as dimensões espaciais (largura e altura) dos

⁴O nome sugere uma operação inversa à convolução, entretanto não é isso que acontece. Esse nome provavelmente veio da ideia de “inverter” a operação de convolução no sentido de expandir as dimensões espaciais da entrada e não diminuir.

Figura 1.25: Arquitetura típica de uma Rede Neural Convolucional com uma camada de convolução e uma camada de *pooling*, em que é enviado para rede a imagem de um cachorro e é feita a classificação do objeto na imagem.



Fonte: Elaboração do Autor.

mapas de características. Essa operação é frequentemente empregada em tarefas de geração de imagens, em que o objetivo é reconstruir imagens a partir de representações compactas. A convolução transposta funciona de maneira inversa à convolução válida. Enquanto a convolução reduz as dimensões espaciais da imagem de entrada, a convolução transposta expande essas dimensões. Isso é alcançado gerando uma saída com dimensões calculadas da seguinte forma:

$$\begin{aligned} w_{out} &= (w_{in} - 1) \cdot s_w - 2w_{pad} + w_{kernel}, \\ h_{out} &= (h_{in} - 1) \cdot s_h - 2h_{pad} + h_{kernel}, \end{aligned} \quad (1.29)$$

em que w_{in} e h_{in} são a largura e a altura da imagem de entrada, s_w e s_h são os valores do *stride* na largura e na altura, w_{pad} e h_{pad} são a quantidade de pixels adicionados como *padding* na largura e na altura (geralmente os valores para o *padding* são zero), e w_{kernel} e h_{kernel} são a largura e a altura do *kernel*, respectivamente. Cada entrada da imagem de saída é calculada como uma soma ponderada dos valores da imagem de entrada multiplicados pelos pesos do *kernel*. A convolução transposta pode ser implementada utilizando o seguinte Algoritmo 4.

A Figura 1.26 exemplifica a operação de convolução transposta, em que uma matriz de entrada X (matriz 2×2) é operada com um *kernel* W (matriz 2×2) para produzir a saída Y (matriz 3×3).

Entrada: X (matriz de entrada), W (*kernel*), s_w (*stride* na largura), s_h (*stride* na altura), w_{pad} (*padding* na largura), h_{pad} (*padding* na altura)

Saída: Y (matriz de saída)

$w_{out} \leftarrow (w_{in} - 1) \cdot s_w - 2w_{pad} + w_{kernel};$

$h_{out} \leftarrow (h_{in} - 1) \cdot s_h - 2h_{pad} + h_{kernel};$

$Y \leftarrow$ matriz de zeros de tamanho $w_{out} \times h_{out};$

Para $i \leftarrow 0$ **Até** $w_{in} - 1$ **Faça**

Para $j \leftarrow 0$ **Até** $h_{in} - 1$ **Faça**

Para $m \leftarrow 0$ **Até** $w_{kernel} - 1$ **Faça**

Para $n \leftarrow 0$ **Até** $h_{kernel} - 1$ **Faça**

$Y[i \cdot s_w + m - w_{pad}][j \cdot s_h + n - h_{pad}] += X[i][j] \cdot W[m][n];$

Fim

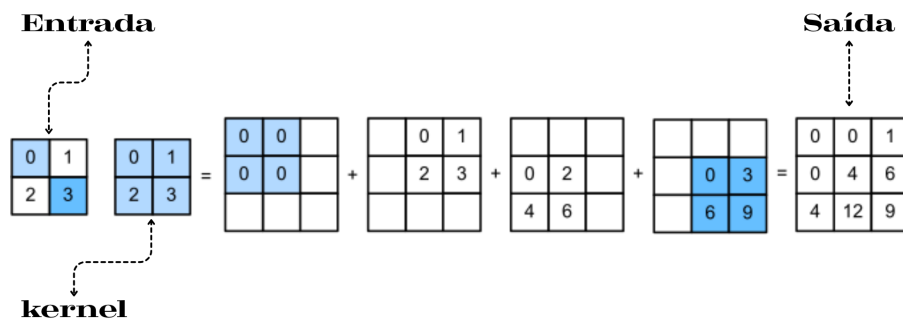
Fim

Fim

Fim

Algoritmo 4: Convolução Transposta

Figura 1.26: Exemplo da operação de convolução transposta.



Fonte: Elaboração do Autor.

1.4 U-net: Rede Convolutacional para Segmentação de Imagens

Modificando a arquitetura tradicional de uma CNN, é possível obter variadas redes neurais com objetivos específicos. O uso típico de redes convolucionais ocorre em tarefas de classificação, em que o objetivo é atribuir uma etiqueta a uma imagem inteira. No entanto, em muitas tarefas visuais, especialmente no processamento de imagens biomédicas, a saída desejada deve incluir informações espaciais detalhadas, como a localização precisa de estruturas dentro da imagem. Para atender a essa necessidade, foi proposta a arquitetura U-Net (RONNEBERGER; FISCHER; BROX, 2015), que é uma rede neural convolutacional projetada para segmentação de imagens. A U-Net é composta por uma parte de contração (*encoder*, composta por camadas de convolução e *pooling*) e uma parte de expansão (*decoder*, composta por camadas de convolução transposta), formando uma estrutura em forma de U (origem do nome U-Net). O *encoder* é responsável por extrair características da imagem de entrada, enquanto o *decoder* reconstrói a imagem, utilizando as características extraídas pelo *encoder*. A arquitetura U-Net é especialmente eficaz em tarefas de segmentação semântica, em que o objetivo é classificar cada pixel da imagem em uma categoria específica, permitindo a identificação precisa de objetos e estruturas dentro da imagem. O Exemplo 1.8 apresenta uma simplificação visual do funcionamento da U-Net antes de ser introduzido os detalhes da arquitetura completa.

Exemplo 1.8. *Seja uma imagem de entrada normalizada X de dimensões 5×5 e uma resposta desejada Y de dimensões 5×5 , a Figura 1.27 apresenta as duas imagens, em que 1 representa a cor mais brilhante e 0 representa o fundo branco.*

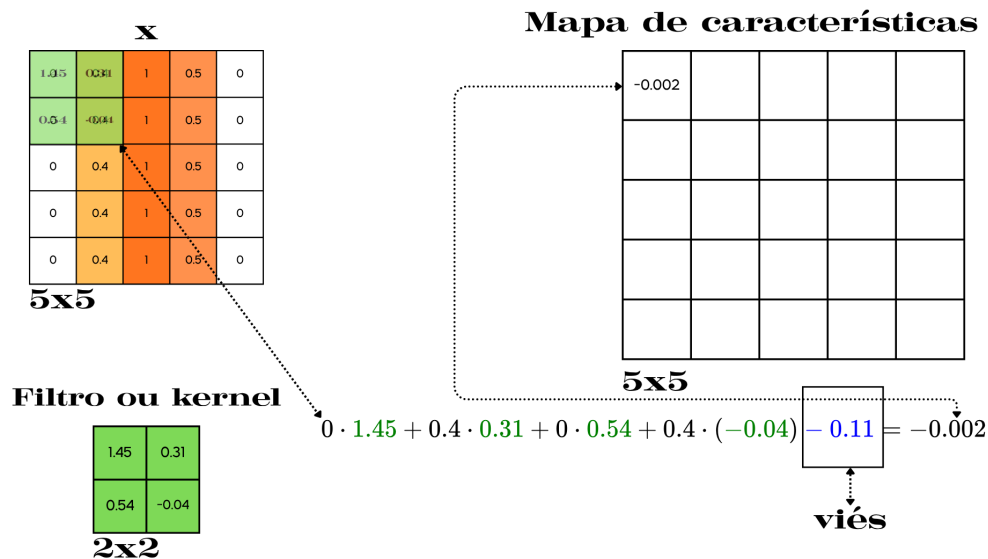
Figura 1.27: Imagem de entrada X e resposta desejada Y .

X					Y - Imagem segmentada				
0	0.4	1	0.5	0	0	1	1	1	0
0	0.4	1	0.5	0	0	1	1	1	0
0	0.4	1	0.5	0	0	1	1	1	0
0	0.4	1	0.5	0	0	1	1	1	0
0	0.4	1	0.5	0	0	1	1	1	0
5x5					5x5				

Fonte: Elaboração do Autor.

Na imagem de saída esperada, pode-se entender os valores de pixel iguais a 1 como regiões de interesse e os valores iguais a 0 como fundo. O primeiro passo para gerar a imagem de saída é aplicar a operação de convolução na imagem de entrada X utilizando um kernel que neste exemplo é uma matriz 2×2 , como mostra a Figura 1.28.

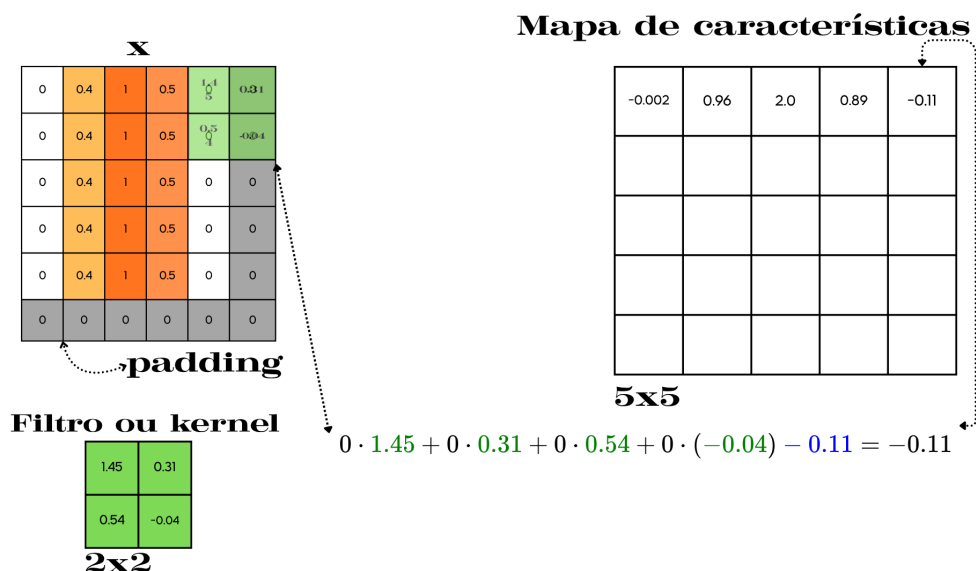
Figura 1.28: Aplicação de um *kernel* de convolução na imagem de entrada X . O resultado da convolução é um mapa de características que destaca as regiões de interesse na imagem.



Fonte: Elaboração do Autor.

Os valores do *kernel* e do viés que apareceram neste exemplo já foram otimizados para destacar as regiões de interesse na imagem. Para que o mapa de características resultante da convolução tenha as mesmas dimensões da imagem de entrada, é necessário aplicar a técnica de *padding*, adicionando uma borda de zeros ao redor da imagem de entrada X , como mostra a Figura 1.29.

Figura 1.29: Aplicação de *padding* na imagem de entrada X para preservar suas dimensões durante a operação de convolução.

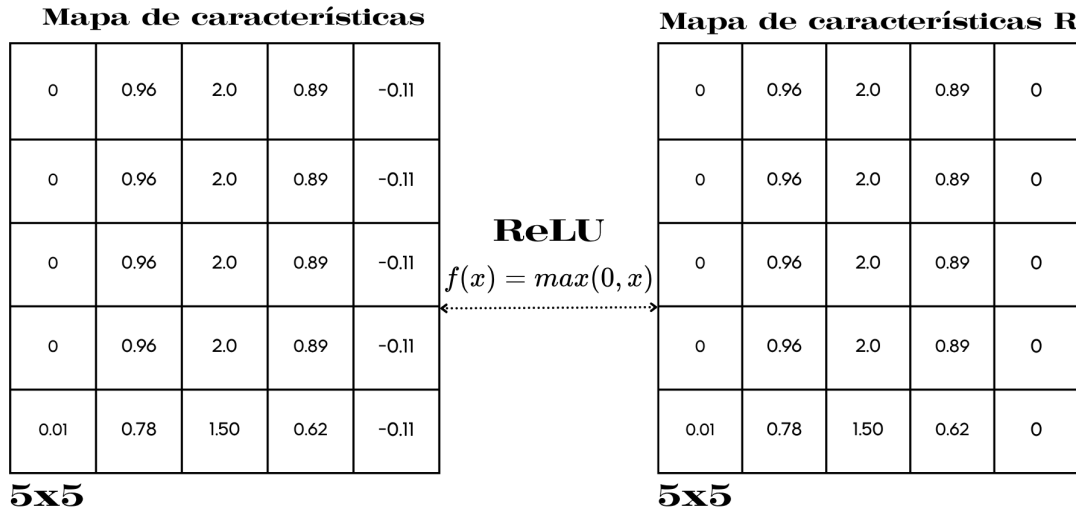


Fonte: Elaboração do Autor.

Após o *kernel* ser aplicado na imagem de entrada X com o uso de *padding*, é gerado um mapa de características, entretanto para completar a operação, é necessário aplicar uma função

de ativação em cada valor do mapa, como mostra a Figura 1.30. A função de ativação utilizada é a função *ReLU*, vista na Equação 1.4.

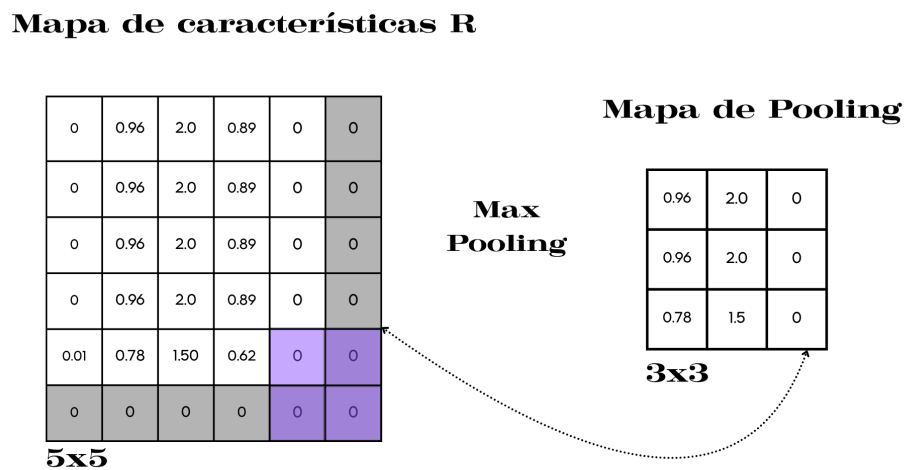
Figura 1.30: Aplicação da função de ativação *ReLU* no mapa de características resultante da convolução.



Fonte: Elaboração do Autor.

O mapa de características resultante da aplicação da função de ativação é então processado por uma camada de *Max pooling* de dimensões 2×2 e *stride* 2, que reduz suas dimensões espaciais pela metade, como mostra a Figura 1.31.

Figura 1.31: Aplicação de uma camada de *Max pooling* no mapa de características resultante.

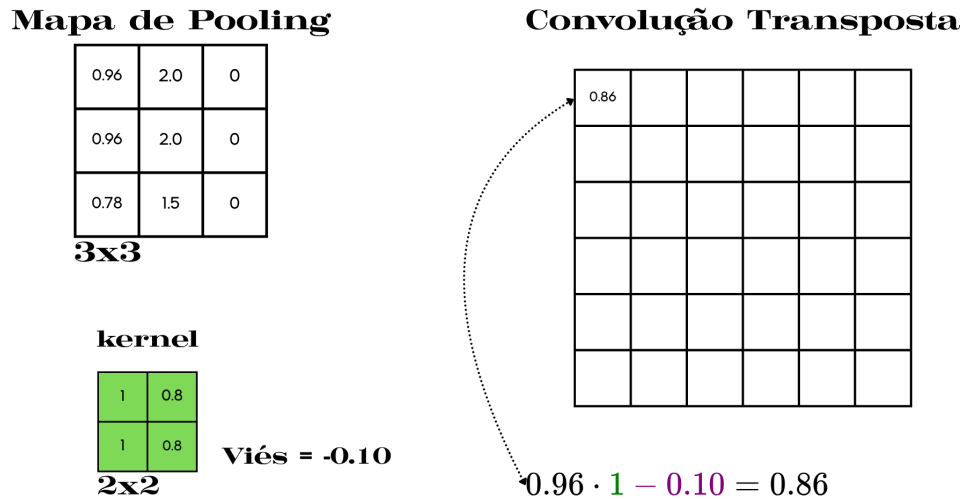


Fonte: Elaboração do Autor.

Com o mapa de pooling gerado, para voltar as dimensões originais da imagem de entrada,

é necessário aplicar a operação de convolução transposta, utilizando um kernel de dimensões 2×2 , como mostra a Figura 1.32.

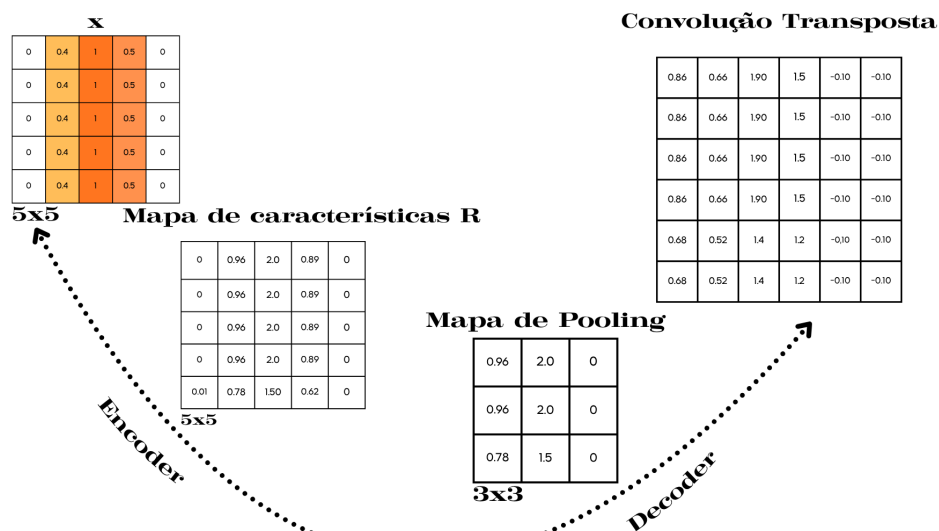
Figura 1.32: Aplicação da operação de convolução transposta no mapa de pooling para expandir suas dimensões espaciais e voltar às dimensões originais da imagem de entrada.



Fonte: Elaboração do Autor.

O nome U-Net vem da forma em U da arquitetura, em que o encoder diminui as dimensões espaciais da imagem de entrada, enquanto o decoder as expande, formando uma estrutura simétrica, como mostra a Figura 1.33.

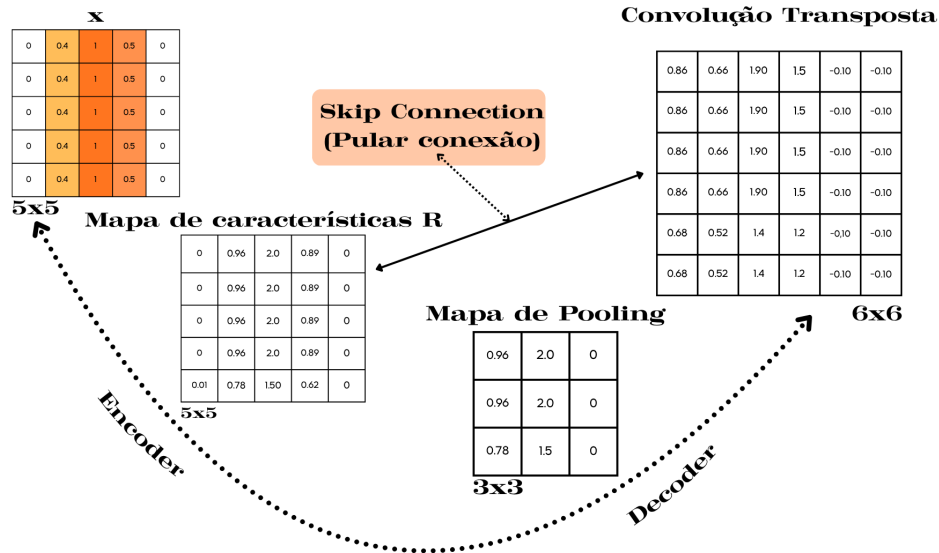
Figura 1.33: Estrutura em forma de U da arquitetura U-Net. O encoder é responsável por extrair características da imagem de entrada, enquanto o decoder reconstrói a imagem, utilizando as características extraídas pelo encoder.



Fonte: Elaboração do Autor.

A característica mais importante da arquitetura U-Net é a presença de conexões de atalho (*skip connections*) entre as camadas correspondentes do encoder e do decoder, presente na Figura 1.34. Essa operação concatena as imagens correspondentes, entretanto para que as ima-

Figura 1.34: Exemplo de conexões de atalho (*skip connections*) na arquitetura U-Net.



Fonte: Elaboração do Autor.

gens possam ser concatenadas, é necessário que elas tenham as mesmas dimensões, portanto, é necessário aplicar a técnica de *copy (copiar) and (e) crop (cortar)*, em que as características extraídas pelo decoder são copiadas e recortadas para se encaixar nas dimensões da camada correspondente no encoder, a sobreposição é feita começando de cima para baixo, da esquerda para a direita, com o vermelho sinalizando a área que será removida após a sobreposição, como mostra a Figura 1.35.

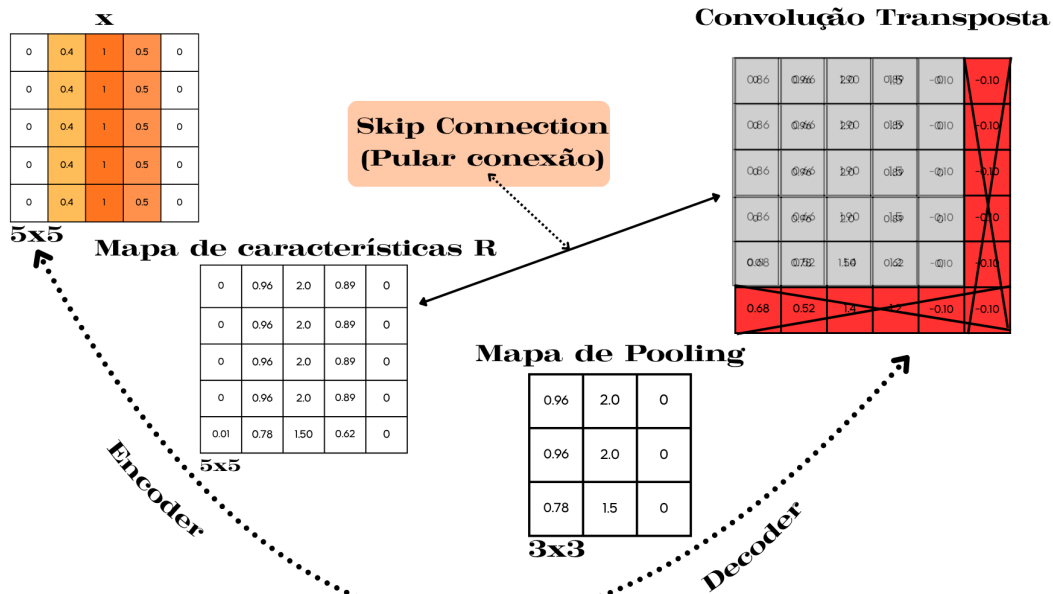
Dependendo da convolução utilizada, o *copy and crop* pode ser aplicado a imagem do encoder e caso as imagens tenham mais de um canal, a operação de concatenação leva em consideração apenas a altura e a largura das imagens, ou seja, as imagens precisam ter as mesmas dimensões, os canais de cada imagem são somados no processo, por exemplo:

$$X_1 = (A, L, C_1) \oplus X_2 = (A, L, C_2) = (A, L, C_1 + C_2) = Y,$$

em que A é a altura, L é a largura e C é a quantidade de canais de cada imagem e $\oplus$ representa a operação de concatenação ao longo do eixo dos canais.

Para finalizar a concatenação, é necessário aplicar uma última camada de convolução. Como as imagens estão sobrepostas, por conta da concatenação, o kernel fica com dimensões $(1 \times 1 \times 2)$, um canal para cada imagem, como mostra a Figura 1.36. No fim da convolução, é necessário aplicar uma função de ativação, neste caso a *sigmoid logística* vista na Equação 1.3, para gerar a imagem de saída Y com valores entre 0 e 1, representando a probabilidade de cada pixel pertencer à classe segmentada. Neste exemplo, quanto mais próximo de 1, mais brilhante é o pixel, indicando uma maior probabilidade de não estar no plano de fundo da imagem, enquanto

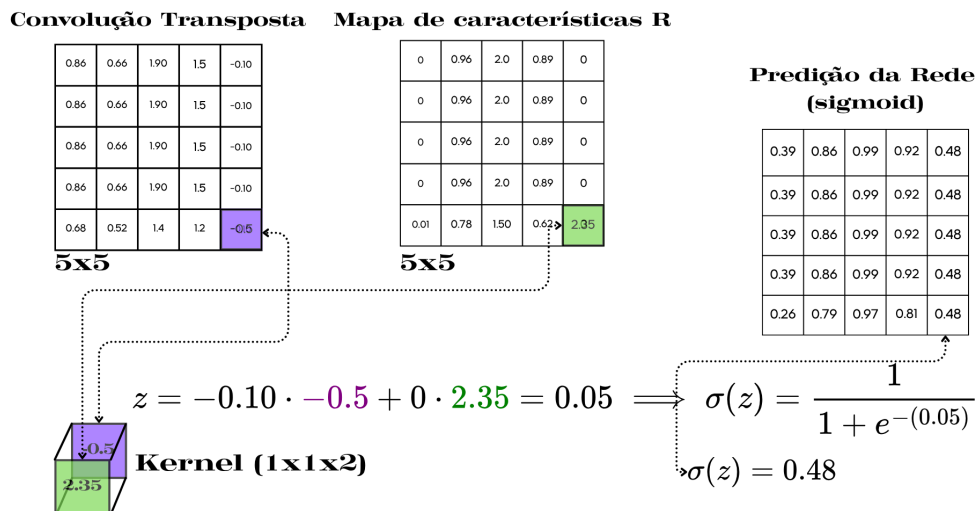
Figura 1.35: Exemplo de conexões de atalho (*skip connections*) na arquitetura U-Net. As conexões de atalho permitem que as informações de baixo nível, como bordas e texturas, sejam diretamente transferidas do encoder para o decoder, facilitando a reconstrução precisa da imagem segmentada.



Fonte: Elaboração do Autor.

valores próximos de 0 indicam uma maior probabilidade de pertecerem ao plano de fundo.

Figura 1.36: Aplicação de uma última camada de convolução para gerar a imagem de saída Y . O *kernel* utilizado tem dimensões $(1 \times 1 \times 2)$, um canal para cada imagem resultante da concatenação.

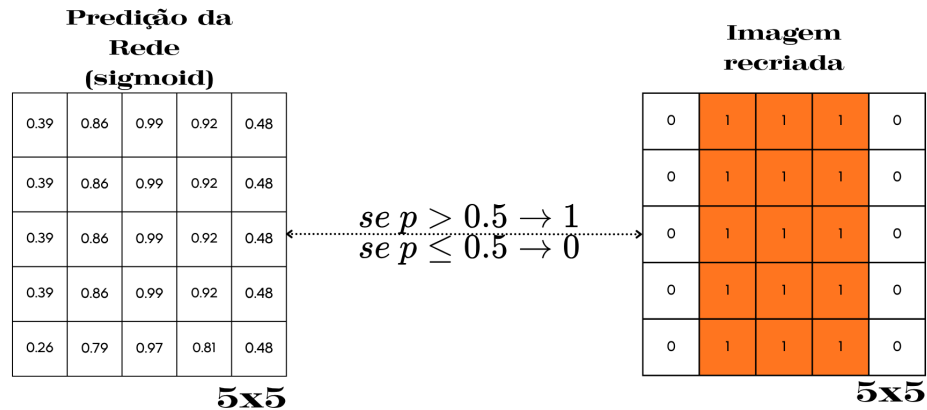


Fonte: Elaboração do Autor.

Por fim, como a imagem segmentada, presente na Figura 1.27, tem valores 0 e 1, é ne-

cessário aplicar um limiar (*threshold*) na imagem gerada após a aplicação da função *sigmoid*. Como mostra a Figura 1.37.

Figura 1.37: Aplicação de um limiar (*threshold*) para gerar a imagem binária final. A imagem segmentada resultante da aplicação da função de ativação *sigmoid* tem valores entre 0 e 1, representando a probabilidade de cada pixel pertencer à classe segmentada. O limiar é aplicado para converter esses valores em uma imagem binária, onde pixels com valores acima do limiar são classificados como pertencentes à classe segmentada (1), enquanto pixels com valores abaixo do limiar são classificados como pertencentes ao plano de fundo (0).



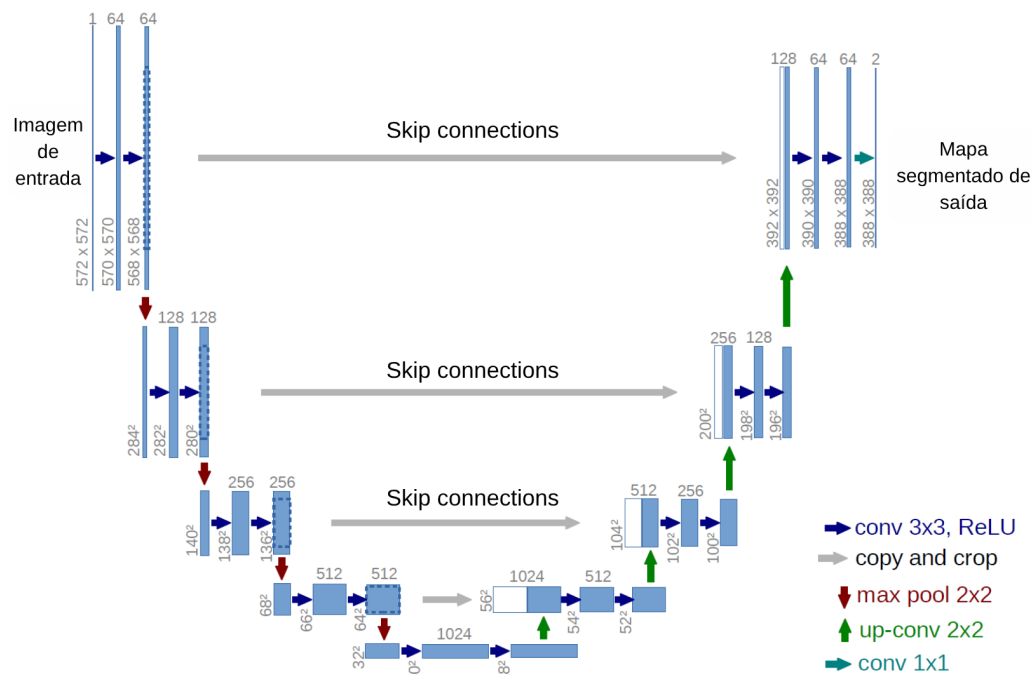
Fonte: Elaboração do Autor.

1.4.1 U-Net

A rede convolucional desenvolvida por Ronneberger et al. (RONNEBERGER; FISCHER; BROX, 2015) é bem similar ao Exemplo 1.8, entretanto, a arquitetura completa da U-Net é composta por múltiplas camadas de convolução e pooling no encoder, e múltiplas camadas de convolução transposta no decoder, no total são quatro camadas de convolução e pooling no encoder e quatro camadas de convolução transposta no decoder, como mostra a Figura 1.38. O número de filtros em cada camada de convolução aumenta à medida que se avança no encoder, começando com 64 filtros na primeira camada e dobrando a quantidade a cada camada subsequente, chegando a 1024 filtros na última camada do encoder. No decoder, o número de filtros é reduzido pela metade a cada camada subsequente, começando com 512 filtros na primeira camada do decoder e terminando com 64 filtros na última camada antes da saída. As conexões de atalho (*skip connections*) entre as camadas correspondentes do encoder e do decoder permitem que sejam feitas as concatenações das imagens. A Figura 1.38 apresenta a arquitetura completa da U-Net, cada bloco representa uma etapa de processamento, o número na parte superior do bloco indica o número de filtros utilizados na camada de convolução, enquanto o número na parte inferior do bloco indica as dimensões espaciais da imagem resultante daquela etapa. A seta azul escuro representa a operação de convolução com filtros de dimensão 3×3 .

e função de ativação *ReLU*, a seta vermelha representa a operação de pooling com *kernel* de tamanho 2×2 e stride de 2, a seta cinza representa a operação de *Copy and Crop* para realizar a concatenação das imagens, e a seta verde representa a operação de convolução transposta com filtros de dimensão 2×2 . A última camada de convolução utiliza um *kernel* de dimensão 1×1 e função de ativação *sigmoid logística* (dependendo da aplicação) para gerar a imagem de saída segmentada.

Figura 1.38: Arquitetura completa da U-Net.



Fonte: Adaptado de (RONNEBERGER; FISCHER; BROX, 2015).

Como não é utilizado nenhum processo de *padding* as dimensões da imagem são reduzidas em cada etapa de convolução. O corte feito para a concatenação (sinalizado por uma linha azul pontilhada) nas imagens do encoder (lado direito da Figura 1.38) é feito no centro da imagem, ou seja, a parte central da imagem do encoder é mantida para a concatenação, enquanto as bordas são removidas. O total de parâmetros a serem otimizados é de aproximadamente 27.548,993 dependendo da utilização do viés ou não. Para as duas primeiras camadas de convolução do encoder o número de parâmetros pode ser calculado da seguinte forma:

$$1^{\text{a}} \text{ convolução} = ((3 \times 3 \times 1) + 1) \times 64 = 640$$

$$2^{\text{a}} \text{ convolução} = ((3 \times 3 \times 64) + 1) \times 64 = 36.928.$$

O filtro da segunda convolução tem 64 canais, um para cada mapa de características gerado pela primeira convolução, multiplica-se novamente por 64, pois cada filtro gera apenas um mapa de características.

1.4.2 DICE

Capítulo 2

Teoria dos Conjuntos Fuzzy

2.1 Teoria dos Conjuntos Fuzzy

A teoria dos conjuntos fuzzy, introduzida por Lotfi A. Zadeh em 1965 (ZADEH, 1965), é uma extensão da teoria clássica dos conjuntos, permitindo a representação de incerteza e imprecisão. Diferentemente dos conjuntos clássicos, onde um elemento pertence ou não a um conjunto, nos conjuntos fuzzy cada elemento possui um grau de pertinência, variando entre 0 e 1. Essa abordagem é especialmente útil em situações onde as fronteiras entre categorias não são claramente definidas, permitindo uma modelagem mais flexível e realista de fenômenos complexos. As definições e propriedades dos conjuntos fuzzy presentes neste capítulo foram baseadas em (JAFELICE; BARROS; BASSANEZI, 2023).

2.1.1 Conjuntos Fuzzy

Um *subconjunto fuzzy* A de um conjunto universo X é definido por uma função de pertinência μ_A que atribui a cada elemento $x \in X$ um valor $\mu_A(x) \in [0, 1]$, representando o grau de pertencimento de x ao conjunto A . Assim:

$$\mu_A : X \rightarrow [0, 1].$$

O valor $\mu_A(x) = 0$ indica que x não pertence ao conjunto A , enquanto $\mu_A(x) = 1$ indica que x pertence completamente ao conjunto. Valores intermediários representam graus parciais de pertencimento, permitindo uma representação mais rica e flexível de conceitos vagos ou imprecisos.

Exemplo 2.1. Considere o conjunto universo X representando a temperatura em graus Celsius, e o conjunto fuzzy A representando a categoria “quente”. A função de pertinência μ_A pode ser

definida da seguinte forma:

$$\mu_A(x) = \begin{cases} 0 & \text{se } 0 \leq x \leq 20 \\ \frac{x-20}{10} & \text{se } 20 < x < 30 \\ 1 & \text{se } x \geq 30 \end{cases}$$

Neste exemplo, temperaturas de 20°C ou menos não são consideradas quentes ($\mu_A(x) = 0$), enquanto temperaturas de 30°C ou mais são consideradas completamente quentes ($\mu_A(x) = 1$). Temperaturas entre 20°C e 30°C têm um grau de pertinência que varia linearmente entre 0 e 1, refletindo a natureza gradual do conceito de “quente”.

2.1.2 Operações entre Conjuntos Fuzzy

As operações entre conjuntos fuzzy, como união, interseção e complemento, são definidas de maneira a preservar a estrutura fuzzy. A união de dois conjuntos clássicos A e B é dada por:

$$A \cup B = \{x \in U; x \in A \text{ ou } x \in B\},$$

enquanto a interseção é definida por:

$$A \cap B = \{x \in U; x \in A \text{ e } x \in B\}.$$

O complemento de um conjunto fuzzy A é dado por:

$$\bar{A} = \{x \in U; x \notin A\}.$$

Essas três operações têm respectivamente as funções características, $\forall x \in X$

$$\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x)), \quad \mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x)), \quad \mu_{\bar{A}}(x) = 1 - \mu_A(x).$$

Definição 2.1. *Sejam A e B conjuntos fuzzy em um conjunto universo X . A união, interseção e complemento de A e B , denotadas por $A \cup B$, $A \cap B$ e $\bar{A}$, respectivamente, são as funções de pertinência, presentes na Figura 2.1, definidas por:*

$$\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x)) \quad \text{para todo } x \in X,$$

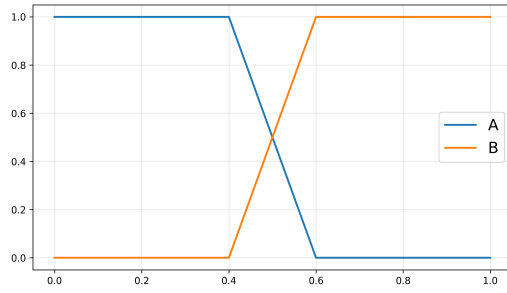
$$\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x)) \quad \text{para todo } x \in X,$$

$$\mu_{\bar{A}}(x) = 1 - \mu_A(x) \quad \text{para todo } x \in X.$$

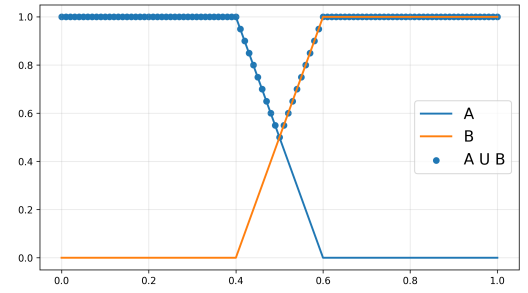
2.1.3 Normas Triangulares

As normas triangulares são generalizações dos operadores união e intersecção.

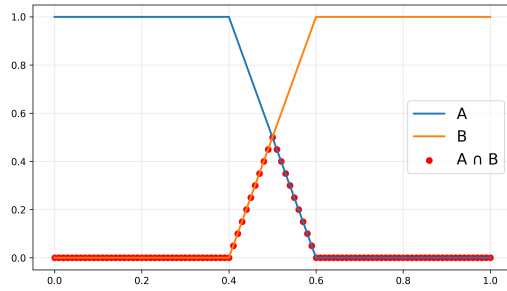
Figura 2.1: Operações entre conjuntos fuzzy.



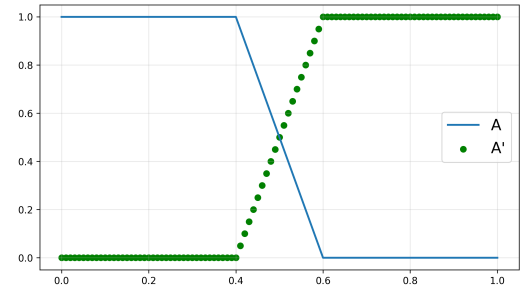
(a) Conjuntos fuzzy A e B.



(b) União e interseção de conjuntos fuzzy A e B.



(c) Interseção de conjuntos fuzzy A e B.



(d) Complemento de um conjunto fuzzy A.

Fonte: Elaboração do Autor.

Definição 2.2. Uma co-norma triangular (*s-norma*) é uma operação binária $S : [0, 1] \times [0, 1] \rightarrow [0, 1]$ que satisfaz as seguintes propriedades:

- *Comutatividade:* $xSy = ySx$
- *Associatividade:* $xS(ySz) = (xSy)Sz$
- *Monotonicidade:* Se $x \leq x'$ e $y \leq y'$, então $xSy \leq x'Sy'$
- *Condição de fronteira:* $xS0 = x$ e $xS1 = 1$

Exemplo 2.2. Soma Limitada: $S : [0, 1] \times [0, 1] \rightarrow [0, 1]$ Com $xSy = \min(1, x + y)$

Definição 2.3. Uma norma triangular (*t-norma*) é uma operação binária $T : [0, 1] \times [0, 1] \rightarrow [0, 1]$ que satisfaz as seguintes propriedades:

- *Comutatividade:* $xTy = yTx$
- *Associatividade:* $xT(yTz) = (xTy)Tz$
- *Monotonicidade:* Se $x \leq x'$ e $y \leq y'$, então $xTy \leq x'Ty'$
- *Condição de fronteira:* $0Tx = 0$ e $1Tx = x$

Exemplo 2.3. Diferença Limitada: $T : [0, 1] \times [0, 1] \rightarrow [0, 1]$ Com $xTy = \max(0, x + y - 1)$

2.1.4 Níveis de um Conjunto Fuzzy

Definição 2.4. *Sejam A um conjunto fuzzy e $\alpha \in (0, 1]$. O α -nível de A , denotado por $[A]^\alpha$, é o conjunto clássico definido por:*

$$[A]^\alpha = \{x \in X \mid \mu_A(x) \geq \alpha\}.$$

Definição 2.5. *O nível zero de um conjunto fuzzy A do universo X , em que X é um espaço topológico, é o fecho topológico do suporte de A , ou seja:*

$$[A]^0 = \overline{\text{supp}(A)}.$$

Definição 2.6. *O suporte de um conjunto fuzzy A são todos os elementos de X que têm grau de pertinência maior que zero em A , ou seja:*

$$\text{supp}(A) = \{x \in X \mid \mu_A(x) > 0\}.$$

Exemplo 2.4. *Seja A o conjunto fuzzy representando a categoria “quente” do Exemplo 2.1, com a função de pertinência definida por:*

$$\mu_A(x) = \begin{cases} 0 & \text{se } x \leq 20 \\ \frac{x-20}{10} & \text{se } 20 < x < 30 \\ 1 & \text{se } x \geq 30. \end{cases}$$

O α -nível de A para $\alpha = 0,5$ é dado por:

$$[A]^{0,5} = \{x \in \mathbb{R} \mid \mu_A(x) \geq 0,5\}.$$

Neste caso, tem-se:

$$\mu_A(x) \geq 0,5 \implies \frac{x-20}{10} \geq 0,5 \implies x \geq 25.$$

Portanto, o α -nível de A para $\alpha = 0,5$ é o intervalo $[25, \infty)$, representando as temperaturas que são consideradas “quentes” com um grau de pertinência de pelo menos 0,5. O suporte de A é dado por:

$$\text{supp}(A) = \{x \in \mathbb{R} \mid \mu_A(x) > 0\}.$$

Neste caso, tem-se:

$$\mu_A(x) > 0 \implies \frac{x-20}{10} > 0 \implies x > 20.$$

Portanto, o suporte de A é o intervalo $(20, \infty)$, representando as temperaturas que têm um grau de pertinência maior que zero em A , ou seja, as temperaturas que são consideradas “quentes” em algum grau.

2.1.5 Números Fuzzy

Um número fuzzy é um subconjunto fuzzy do conjunto dos números reais, caracterizado por uma função de pertinência que atribui a cada número real um grau de pertencimento. Os números fuzzy são frequentemente utilizados para representar quantidades imprecisas ou vagamente definidas, como “muito quente” ou “pouco tempo”.

Definição 2.7. *Um conjunto fuzzy N é chamado de número fuzzy quando o conjunto universo, em que N é definido, é o conjunto dos números reais $\mathbb{R}$, e a função de pertinência μ_N é tal que:*

- $\mu_N(x) = 1$ para pelo menos um valor x de $\text{supp}(N)$;
- $[N]^\alpha$ é um intervalo fechado para todo $\alpha \in (0, 1]$;
- O suporte de N é limitado.

Exemplo 2.5. *Um número fuzzy A é dito trapezoidal se o gráfico de sua função de pertinência tem a forma de um trapézio. A função de pertinência de um número fuzzy trapezoidal pode ser definida por quatro parâmetros a, b, c, d (com $a \leq b \leq c \leq d$) da seguinte forma:*

$$\mu_A(x) = \begin{cases} \frac{x-a}{b-a}, & \text{se } a < x < b \\ 1, & \text{se } b \leq x < c \\ \frac{d-x}{d-c}, & \text{se } c \leq x < d \\ 0, & \text{caso contrário} \end{cases}$$

Neste exemplo, o número fuzzy A é representado por um trapézio definido pelos parâmetros a, b, c, d . Para $a < x < b$, a função de pertinência aumenta linearmente de 0 a 1, indicando um aumento gradual no grau de pertencimento. Para $b \leq x < c$, a função de pertinência é constante em 1, indicando que esses valores pertencem completamente ao conjunto fuzzy. Para $c \leq x < d$, a função de pertinência diminui linearmente de 1 a 0, indicando uma diminuição gradual no grau de pertencimento.

2.2 Sistema Baseado em Regras Fuzzy

Um sistema Baseado em Regras Fuzzy (SBRF) (JAFELICE; BARROS; BASSANEZI, 2023) é um sistema que utiliza a teoria dos conjuntos fuzzy para produzir saídas a partir de entradas dadas por conjuntos fuzzy. Tais sistemas foram desenvolvidos com o objetivo de imitar o comportamento humano considerando que o conhecimento e as ações humanas são apoiadas em uma sequência de regras linguísticas, traduzidas por um conjunto de regras. As regras formuladas pelo ser humano são usualmente da forma:

SE (um conjunto de condições é satisfeito) *ENTÃO* (um conjunto de consequências pode ser inferido).

Definição 2.8. *Uma relação fuzzy R é um subconjunto fuzzy do produto cartesiano entre conjuntos universos $X_1, X_2, \dots, X_n$, ou seja, $R \subseteq X_1 \times X_2 \times \dots \times X_n$. O produto cartesiano fuzzy $A_1 \times A_2 \times \dots \times A_n$ dos subconjuntos fuzzy $A_1, A_2, \dots, A_n$ de $X_1, X_2, \dots, X_n$, respectivamente, é a relação fuzzy R cuja função de pertinência é dada por:*

$$\mu_R(x_1, x_2, \dots, x_n) = \min(\mu_{A_1}(x_1), \mu_{A_2}(x_2), \dots, \mu_{A_n}(x_n)),$$

2.2.1 Regras e Inferência Fuzzy

Uma regra fuzzy é uma sentença da forma “SE X é A ENTÃO Y é B ”, em que A e B são conjuntos fuzzy em X e Y , respectivamente. Tal regra pode ser interpretada como uma relação fuzzy R entre os conjuntos A e B cuja a função de pertinência $\mu_R(x, y)$ depende de $\mu_A(x)$ e $\mu_B(y)$ para cada $(x, y) \in X \times Y$. Em geral utiliza-se a função:

$$\mu_R(x, y) = \min(\mu_A(x), \mu_B(y)).$$

Uma variável linguística é uma variável cujo o valor é expresso qualitativamente por termos linguísticos, que são representados por conjuntos fuzzy e quantitativamente por uma função de pertinência. Por exemplo, a variável linguística “temperatura” pode assumir os termos linguísticos “fria”, “morna” e “quente” representados por conjuntos fuzzy com funções de pertinência que descrevem o grau de pertencimento de cada temperatura a esses termos.

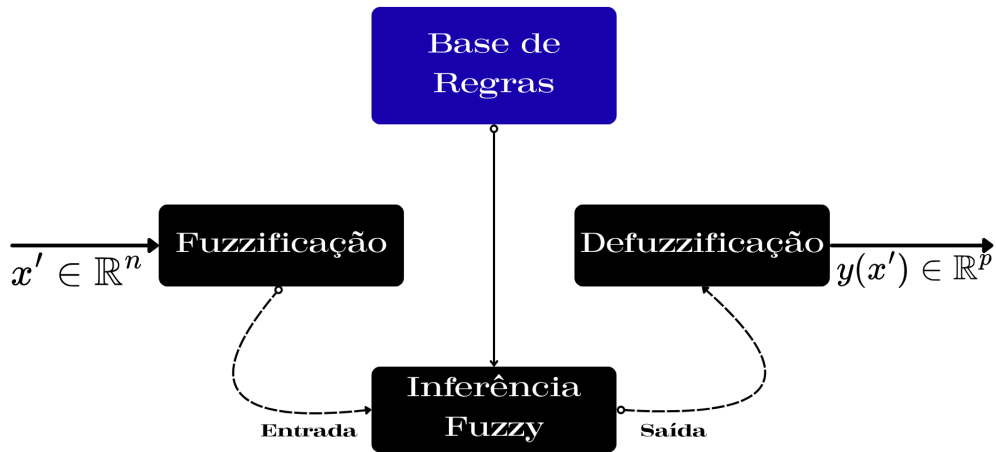
2.2.2 Sistema Baseado em Regras Fuzzy

Um SBRF é composto por quatro componentes: um processador de entrada que realiza a fuzzificação dos dados de entrada, uma coleção de regras, chamada base de regras, uma máquina de inferência fuzzy e um processador de saída que fornece um número real como saída. Estes componentes são conectados como mostrado na Figura 2.2.

O SBRF pode ser visto com um mapeamento entre a entrada e a saída da forma $y = f(x)$, $x \in \mathbb{R}^n$ e $y \in \mathbb{R}^p$. Os componentes do SBRF realizam as seguintes funções:

- O **processador de entrada** (fuzzificação) realiza a fuzzificação dos dados de entrada, ou seja, traduz em conjuntos fuzzy os valores de entrada. A atuação de um especialista é necessária para definir as funções de pertinência que serão utilizadas na fuzzificação, bem como os termos linguísticos que serão associados a cada variável de entrada.
- A **base de regras** contém um conjunto de regras fuzzy que descrevem o comportamento do sistema, essas regras são formuladas por especialistas com base em seu conhecimento sobre o domínio do problema. A base de regras descreve relações entre as variáveis linguísticas para serem utilizadas pela máquina de inferência fuzzy.
- A **máquina de inferência fuzzy** fornece a saída a partir de cada entrada fuzzy e da relação definida pela base de regras. A máquina de inferência fuzzy pode ser vista como

Figura 2.2: Arquitetura de um Sistema Baseado em Regras Fuzzy (SBRF).



Fonte: Adaptado de (JAFELICE; BARROS; BASSANEZI, 2023).

um mecanismo de mapeamento entre os conjuntos fuzzy de entrada e os conjuntos fuzzy de saída, utilizando as regras da base de regras para inferir a saída correspondente a cada entrada. Neste trabalho é utilizado o método de inferência de Mamdani, que é um dos métodos mais comuns para sistemas baseados em regras fuzzy com variáveis linguísticas.

- **Método de inferência de Mamdani:** Uma regra *SE* (antecedente) *ENTÃO* (consequente) é definida pelo produto cartesiano fuzzy dos conjuntos fuzzy que compõem o antecedente e o consequente da regra. O método de Mamdani agrega as regras através do operador lógico *OU*, que é modelado pelo operador do máximo e, em cada regra, o operador lógico *E* é modelado pelo operador do mínimo. As regras são:

Regra 1 : SE (x é A_1 E y é B_1) ENTÃO (z é C_1);

Regra 2 : SE (x é A_2 E y é B_2) ENTÃO (z é C_2).

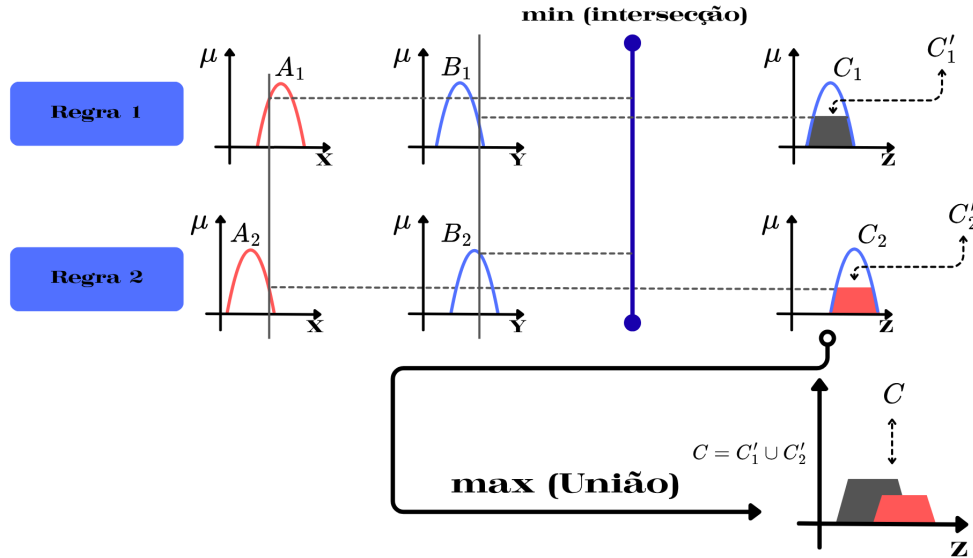
A Figura 2.3 ilustra um exemplo do método de Mamdani, em que duas regras são aplicadas para inferir a saída z a partir das entradas x e y .

Com base no Exemplo 2.1 uma regra que poderia ser formulada utilizando o método de Mamdani é a seguinte:

Regra 1 : SE (x é A) ENTÃO (z é C),

em que A é o conjunto fuzzy representando a categoria “quente” e C é um conjunto fuzzy representando a categoria “ligar o ar condicionado”. A função de pertinência de C poderia ser definida por um número fuzzy trapezoidal, indicando o grau de pertencimento de cada valor de z à categoria “ligar o ar condicionado”. Assim, a regra estabelece que se

Figura 2.3: Exemplo de inferência fuzzy utilizando o método de Mamdani.



Fonte: Adaptado de (JAFELICE; BARROS; BASSANEZI, 2023).

a temperatura x é considerada quente (com um certo grau de pertinência), então a ação z de ligar o ar condicionado também tem um certo grau de pertinência.

- O **processador de saída** (defuzzificação) realiza a defuzzificação, ou seja, converte os conjuntos fuzzy de saída em números reais. Em sistemas fuzzy, em geral a saída é um conjunto fuzzy. Assim, devemos escolher um método de defuzzificação para obter um número real a partir do conjunto fuzzy de saída. O método de defuzzificação mais utilizado é o método de Centro de gravidade, que calcula o centro de massa do conjunto fuzzy de saída para obter um valor real representativo da saída do sistema.

- **Método de Centro de Gravidade:** O método de centro de gravidade é semelhante a média ponderada para distribuições de dados, com a diferença que os pesos são os valores $C(z_i)$ que indicam o grau de compatibilidade do valor z_i com o conceito modelado pelo conjunto fuzzy C . Para domínios discretos, o método de centro de gravidade é dado por:

$$G(C) = \frac{\sum_{i=0}^n z_i \times C(z_i)}{\sum_{i=0}^n C(z_i)}.$$

Referências Bibliográficas

- BEAR, M. F.; CONNORS, B. W.; PARADISO, M. A. *Neurociências: desvendando o sistema nervoso*. 4. ed. Porto Alegre: Artmed, 2017. ISBN 9788582713091.
- BENGIO, Y. Practical recommendations for gradient-based training of deep architectures. *Cornell University*, Version 2, 2012.
- BOOK, D. L. *Capítulo 10: As Principais Arquiteturas de Redes Neurais*. 2025. Acessado: 2026-03-26. Disponível em: <https://www.deeplearningbook.com.br/>.
- BOOK, D. L. *Capítulo 19: Overfitting e Regularização: Parte 1*. 2025. Acessado: 2026-01-26. Disponível em: <https://www.deeplearningbook.com.br/overfitting-e-regularizacao-parte-1/>.
- BOOK, D. L. *Capítulo 22 – Regularização L1*. 2025. Acessado: 2026-03-26. Disponível em: <https://www.deeplearningbook.com.br/>.
- CREVIER, D. *AI: The Tumultuous History of the Search for Artificial Intelligence*. New York: Basic Books, 1993. ISBN 0-465-02997-3.
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. [S.l.]: MIT Press, 2016. <http://www.deeplearningbook.org>.
- HECHT-NIELSEN, R. Theory of the backpropagation neural network. *Academic Press*, 1992.
- JAFELICE, R. S. M.; BARROS, L. C.; BASSANEZI, R. C. *Teoria dos Conjuntos Fuzzy com Aplicações*. 3. ed. São Paulo -SP, Brasil: SBMAC, 2023. ISBN 978-65-86399-14-5.
- LECUN, Y. *The MNIST Database of Handwritten Digits*. 1998. Acessado: 2026-02-22. Disponível em: <http://yann.lecun.com/exdb/mnist/>.
- MCCULLOCH, W.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 1943.
- MÜLLER, V. C. Review of margaret boden, “mind as machine: A history of cognitive science”. *Minds and Machines*, Springer, v. 18, n. 1, p. 121–125, 2008.
- O’SHEA, K.; NASH, R. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015. Disponível em: <https://arxiv.org/abs/1511.08458>.
- RONNEBERGER, O.; FISCHER, P.; BROX, T. U-net: Convolutional networks for biomedical image segmentation. In: *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*. Munich, Germany: Springer, 2015. p. 234–241.
- ROSENBLATT, F. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 1958.

- RUDER, S. An overview of gradient descent optimization algorithms. *Insight Centre for Data Analytics, NUI Galway Aylien Ltd., Dublin*, 2016.
- SOVIERZOSKI, M. A. Convolução de sinais: definição, propriedades e ferramentas. *Revista Ilha Digital*, v. 2, p. 81–95, 2010. ISSN 2177-2649. Disponível em: <https://ilhadigital.florianopolis.ifsc.edu.br/index.php/ilhadigital>.
- STANFIELD, C. L. *Fisiologia Humana*. São Paulo: Pearson Education do Brasil, 2014. ISBN 978-85-430-1426-5.
- SUTSKEVER, I. On the importance of initialization and momentum in deep learning. *International Conference on Machine Learning*, 2013.
- SZELISKI, R. *Computer Vision: Algorithms and Applications*. Cham, Suíça: Springer, 2010. ISBN 978-3030343712.
- TREVISAN, V. *Como funcionam as Redes Neurais Convolucionais (CNNs)*. 2021. Acessado: 2025-12-16. Disponível em: <https://medium.com/data-hackers/como-funcionam-as-redes-neurais-convolucionais-cnns-71978185c1>.
- WERBOS, P. J. Backpropagation through time: What it does and how to do it. *PROCEEDINGS OF THE IEEE*, 1990.
- ZADEH, L. A. Fuzzy sets. *Information and Control*, v. 8, n. 3, p. 338–353, 1965.